Qualcomm Technologies, Inc.

SW6100/SW6100P Sensors Guide - Linux Android Wear

80-74841-90 AB

April 21, 2026

About Qualcomm

Qualcomm relentlessly innovates to deliver intelligent computing everywhere, helping the world tackle some of its most important challenges. Building on our 40 years of technology leadership in creating era-defining breakthroughs, we deliver a broad portfolio of solutions built with our leading-edge AI, high-performance, low-power computing, and unrivaled connectivity. Our Snapdragon® platforms power extraordinary consumer experiences, and our Qualcomm Dragonwing™ products empower businesses and industries to scale to new heights. Together with our ecosystem partners, we enable next-generation digital transformation to enrich lives, improve businesses, and advance societies. At Qualcomm, we are engineering human progress.

Contents

1 SW6100/SW6100P sensors and sensing hub overview

This section introduces the SW6100/SW6100P sensors architecture and core components. It describes how sensors integrate across subsystems, how the Qualcomm® sensing hub (QSH) works, and what features and platforms are supported.

The Qualcomm® system-on-chip (SoC) includes an application processor that runs the Linux operating system, a low-power application digital signal processor (aDSP), and two Cortex M55 Microcontrollers: sensor wear microcontroller (SWM) and ambient wear microcontroller (AWM). Both SWM and AWM run the Zephyr operating system. SWM handles sensor data streaming using the QSH framework. Most of the physical sensors connect to the SWM. AWM provides display watchface rendering functionality for the Ambient mode.

The aDSP runs the Qualcomm Real Time (QuRT™) OS to handle sensor data streaming using the QSH framework. The aDSP supports the following:

- GPIOs configurable as serial bus: serial peripheral interface (SPI), inter-integrated circuit (I^2C), improved I^2C (I^3C), and universal asynchronous receiver/transmitter (UART).
- Serial buses in low-power mode.
- Dedicated local memory, also known as the Island in QSH.

1.1 SW6100/SW6100P sensors architecture

The Qualcomm® sensing hub (QSH) functionality and data paths span multiple subsystems to efficiently support a variety of use cases in low-power mode. The following diagram depicts a high-level sensors architecture, followed by a table that describes each block of architecture.

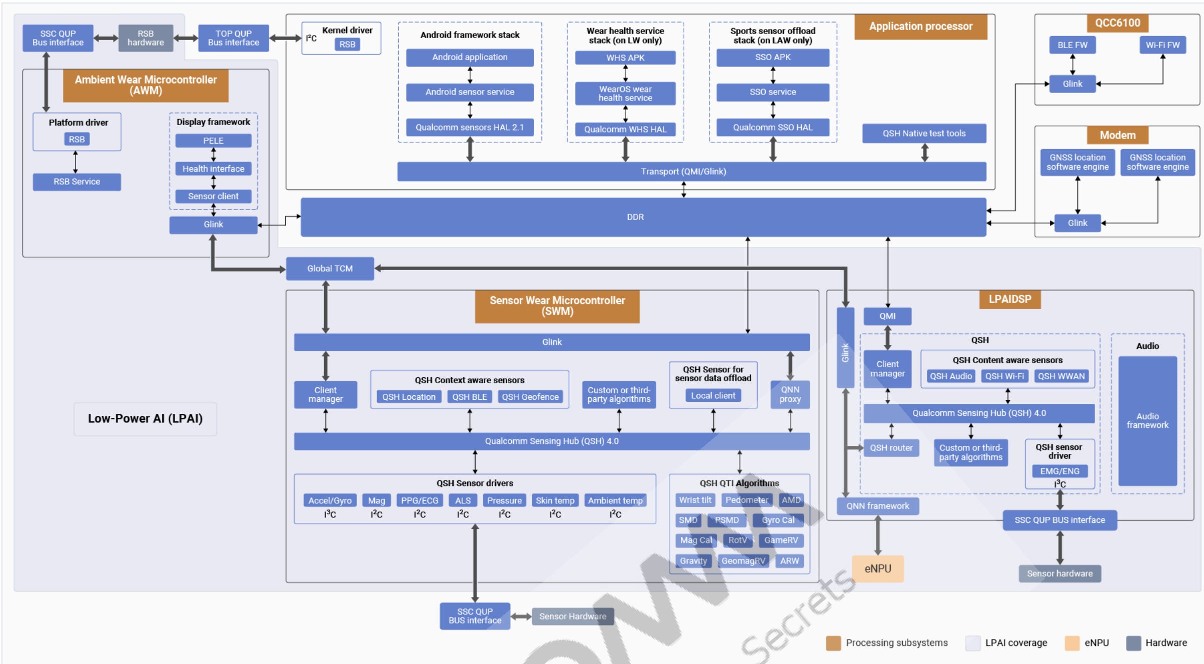


Figure: Sensors architecture

QSH subsystems

Subsystem	Description
Application processor	• **Android app:** An Android application that utilizes sensor data using the Android framework APIs. • **Android sensor service:** Invokes the sensor hardware abstraction layer (HAL) interfaces to provide sensor functionality to the Android framework application/client. • **Sensor HAL:** Provides interfaces to the Android sensor service for sensor operations and manages interactions with QSH. • **Wear health service (WHS) HAL on Linux wearables (LW) (on LW only):** Provides interfaces to the WHS service and manages the WHS interactions with QSH. • **Sports sensor offload (SSO) HAL on Linux Android wearables (LAW) (on LAW only):** Provides sensor data offload capability for rendering on the display watchface. • **QSH native test tools:** Set of tools to test the sensor functionality. • **Rotary side button (RSB) sensor driver:** Provides RSB sensor scroll rotation data in Interactive mode.
Low-power AI (LPAI)	• LPAI provides various capabilities and consists of multiple subsystems that operate in lower-power mode than the application processor. • LPAI can perform operations independently until there is a need for application processor-based functionality.

Subsystem	Description
Sensor wear microcontroller (SWM)	• **Physical sensors:** Most physical sensors connect to SWM. • **QTI algorithms:** QTI provides algorithms for various activities, such as detect device orientation, activity recognition, and step count. • **QSH location:** Provides GPS position fixes and measurements/clock information events. • **QSH geofence:** Provides a way to set up geo boundaries using latitude, longitude, and radius to receive events when the device breaches the geofence. • **QSH Bluetooth low energy (BLE):** Provides BLE advertising and BLE scanning functionality. • **Qualcomm® neural network (QNN) proxy:** Provides access to the embedded neural processor unit (eNPU) hardware using the QNN framework on LPAIDSP. This is useful for the AI use cases.
LPAI digital signal processor (LPAIDSP)	• **Physical sensors:** Electromyography / Electroneurography (EMG/ENG) connect to the LPAIDSP. • **QSH audio:** Provides audio events processing for low power use cases in the Island mode. • **QSH WWAN:** Provides information about the serving and neighboring cells of a mobile device. • **QSH Wi-Fi:** Provides the Wi-Fi scan information. • **QNN framework:** Provides access to the eNPU hardware. • **Audio framework:** Provides information for QSH audio.
Ambient wear microcontroller (AWM)	• **Display framework:** Provides display watchface rendering functionality for the Ambient mode. • **RSB sensor driver:** Provides RSB sensor scroll rotation data in Ambient mode.
Modem	• **GNSS location software engine:** Provides information for QSH location. • **Geofence software engine:** Provides information for QSH geofence.
QCC6100	• **BLE firmware:** Provides information for QSH BLE. • **Wi-Fi firmware:** Provides information for QSH Wi-Fi and QSH WWAN.

1.2 QSH architecture

This section introduces the QSH framework. For detailed QSH functionality, see the following topics in Sensors Execution Environment (SEE) Sensors Deep Dive.

- SEE
- SEE Sensor API
- SEE Service Manager and Services
- SEE Platform Sensors
- Registry in SEE

The following table describes common QSH terms.

QSH terminology

Term	Description
Sensor	• Produces a single type of data. For example, accelerometer, gyroscope, timer, interrupt, and rotation vector. • Handles asynchronous data. • Publishes mandatory and custom attributes, and manages its instances.
Sensor instance	• Runs at a specific configuration and publishes output data events. • It's either created per client request or shared among multiple requests. • Physical sensors usually share a single instance.
Sensor unique identifier (SUID)	• A unique 128-bit identifier for each sensor.
Service	• A module that provides a synchronous interface for common utilities.
Data stream	• A unique connection between a client and data source.
Request	• A configuration message that a client sends to a sensor (see `sns_request.h` file).
Event	• Asynchronous output data message that a sensor instance generates (see `sns_sensor_event.h` file).
Nanopb	• A small code-size protocol buffer implemented in ANSI-C.

The QSH framework includes the following components:

- Application processor software modules
  - Client application: It interacts with the QSH client APIs on the application processor side.
  - QSH client APIs: It offers high-level APIs to access services offered by QSH. It simplifies application development by abstracting system complexities and focusing on the application logic.
- Low-power processor software modules
  - Client manager: It's in charge of all the communication between low-power processors and application processors.

The following table describes responsibilities of the client manager.

Client manager responsibilities

Function	Description
Translate incoming requests	The client manager takes incoming requests and translates them into a format that the QSH can understand.
Translate outgoing indications	The client manager receives event messages from the QSH and translates these event messages into outgoing indications in a format that's understandable outside the QSH.
Guarantees batching options	If a client specifies certain batching (store/accumulate locally) options, then the client manager ensures that they meet the batching options. The client manager checks that the data is grouped and sent in the same way as the client has specified, ensuring compliance with the criteria.

1.2.1 Sensor and sensor instances

QSH divides the sensor implementation into two logical units: sensor and sensor instance.

- Sensors produce, consume, or both produce and consume asynchronous data.
  - Each sensor can have one or more sensor instances.
  - Any request to a sensor for data results in the creation of a sensor instance or sharing of an existing sensor instance.
- Sensors create sensor instances on demand, manage their lifecycle and configuration, and send configuration updates and initial state events to their clients.
  - Each sensor instance operates with a specific client configuration.
  - A physical sensor instance programs the hardware to operate with the required configuration.
  - Vendors must serve all client requests with a minimal number of sensor instances.
  - The sensor instance generates and streams data to all the active clients.
- Multiple sensors can share and configure a single sensor instance. This mode of operation is typical of combo drivers for hardware sensors, such as:
  - Accelerometer and gyroscope
  - Proximity and ambient light

1.2.2 Communication among sensors

Every sensor algorithm and sensor driver within the QSH framework is called a sensor. Sensors communicate with each other through request and event messages over data streams. The message payloads are defined in the protocol buffer format, using the nanopb generator, encoder, and decoder. The message payload length, message ID, and timestamp (for events) are communicated within the metadata managed by the QSH framework.

The following figure shows the communication between the data client and the data source, using the data stream:

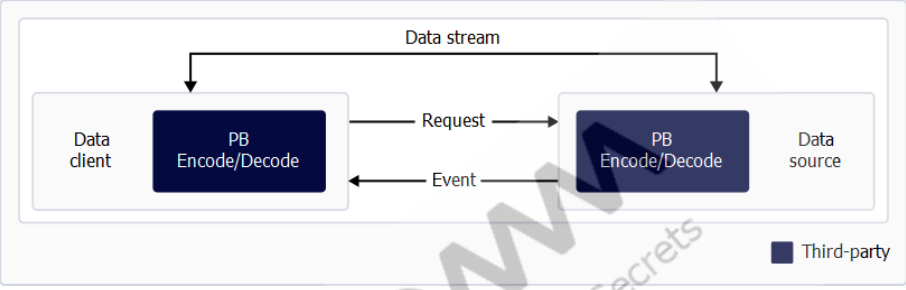


Figure: Sensor communication between client and source

- The client sends request messages to enable, disable, and reconfigure a sensor. Request messages are always addressed to a specific SUID. After the target sensor receives the request message, it sends the request to the sensor instance for proper handling.
- Sensor instances send event messages asynchronously to their registered clients, which can be other sensors or sensor instances.

1.2.3 Nanopb protocol buffer in QSH

QSH uses nanopb protocol buffer for the following:

- All the request and event messages that are exchanged between the sensors.
  - A sensor or sensor instance must encode the payload (if present) for all requests it sends to its dependents.
  - A sensor or sensor instance must decode the payload (if present) for all requests it receives.
  - A sensor or sensor instance must encode the payload (if present) for all events it publishes.
  - A sensor or sensor instance must decode the payload (if present) for all events it receives from its dependents.
- Representing the attribute data
  - All attribute values are in the nanopb-encoded format.
- Diagnostic log packet payload
  - All payloads in the diagnostic log packets are in the nanopb-encoded format.

1.3 Sensing hubs

The presence of two subsystems with unique QSH capabilities requires a unique identifier to communicate and utilize the functionalities. As mentioned in the following table, the communication mechanism for each sensing hub is different.

QSH sensing hubs

Sensing hub	Sensing hub unique ID	Communication mechanism
SWM QSH	Hub 2	Glink
LPAIDSP QSH	Hub 1	QMI

See Sensing hubs, for a list of capabilities of the SWM and LPAIDSP QSH.

The following figure depicts a high-level sensing hub capability discovery and request routing, followed by a brief description.

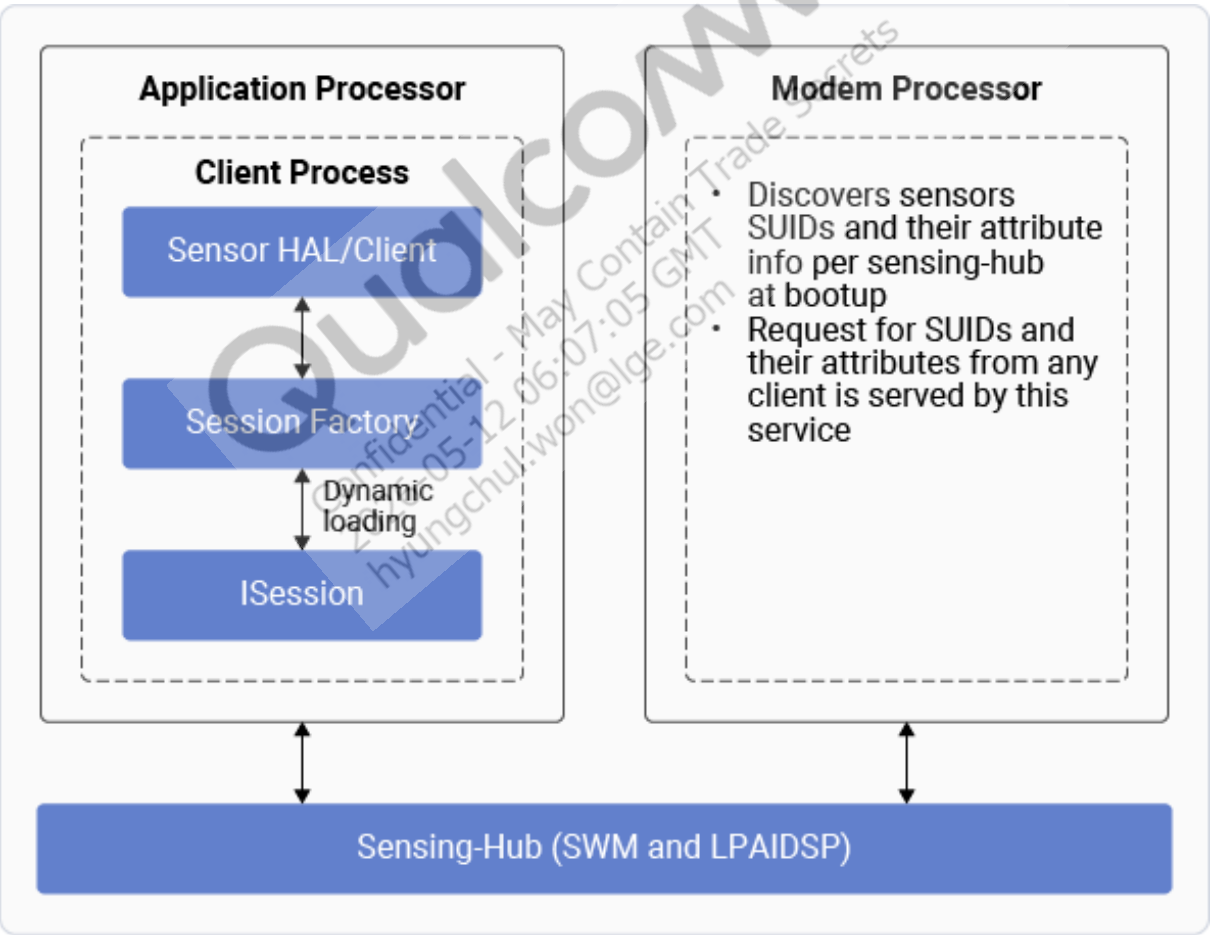


Figure: Sensing hubs

- At device bootup, the Session Factory discovers sensor SUIDs and their attribute information per sensing hub.

- Sensor HAL and ISession native client on the application processor are aware of the sensing hub implementations and automatically use the required communication mechanism.
- The Android framework and Android apps don't to be sensing hub aware, as the Sensor HAL is responsible for routing the client request to the respective hub.
- For a client on a non-application processor, such as the modem processor, it must use the communication mechanism required for the respective sensing hub.
- Initially, the client needs to discover the sensors/SUIDs available on the sensing hub and fetch the sensor attributes. Later, the client can enable or disable the required sensor.

1.4 Registry

See the following resources for details about the registry functionality:

- See Sensors Execution Environment (SEE) - Sensor Config And Registry (English), for the overview video training.
- See the *Registry* section in Sensors Execution Environment (SEE) Sensors Deep Dive, for documentation.

In SW6100/SW6100P, the following locations maintain separate sensor configurations for each sensing hub:

- In HLOS source code: `<hlos_workspace>/vendor/qcom/proprietary/sensors-ship`.
- On the device: `<hlos_workspace>/vendor/qcom/proprietary/sensors-ship`.

1.5 QSH Router

Algorithms integrated in the LPAIDSP QSH depend on the physical sensor data or algorithm data from the SWM QSH. To facilitate this data routing, LPAIDSP QSH provides the QSH router as part of the QSH framework. The QSH router is transparent and doesn't require any special configuration for the algorithm integrated in the LPAIDSP QSH.

The QSH router classifies the QSH sensor as:

- Local sensor, which is local to the sensing hub.
- Remote sensor, which is available on another sensing hub.

> **Note:**
>
> QSH router enables only one-way access, that is, SWM QSH sensors (sensor drivers, algorithms, and context-aware sensors) are accessible by the LPAIDSP QSH algorithms. Whereas the SWM QSH can't access the LPAIDSP QSH sensors.

The following figure provides an overview of the QSH router design.

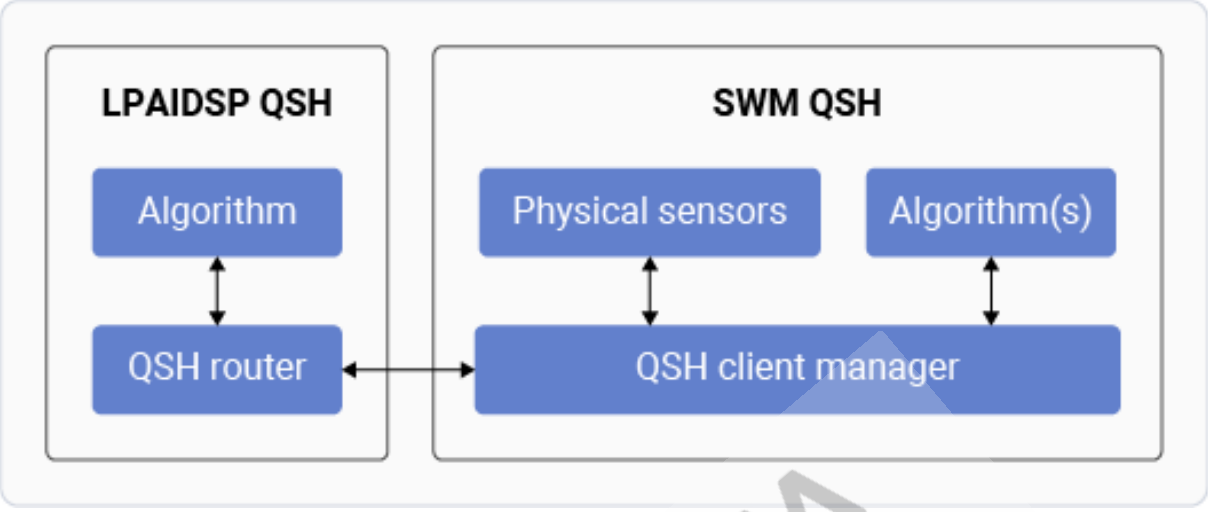


Figure: QSH router

1.6 Sensors features

This section describes capabilities of various sensors-related features.

1.6.1 Low-power AI (LPAI) processing engines

LPAI modules

Module	Capabilities
Processing units	• 2 Cortex M55 microcontroller unit (MCU): sensor wear microcontroller (SWM) and ambient wear microcontroller (AWM) • 1 Qualcomm® Hexagon Processor v79: LPAI digital signal processor (LPAIDSP)
LPAI Global tightly coupled memory (TCM)	• 3.5 MB shared memory between LPAIDSP, SWM, AWM, and eNPU
Important functionalities	• Sensors and algorithms • Audio • Embedded neural processing unit (eNPU) / AI • Display
Important hardware blocks	• eNPU6 • Low-power Island (LPI) GPIOs • Qualcomm® Snapdragon™ sensors core Qualcomm universal peripherals (SSC QUPs): 14 serial engines (6 are dedicated for sensors)

1.6.2 Sensing hubs

SWM and LPAIDSP subsystems

Module	**Capabilities**	
	SWM	**LPAIDSP**
CPU / Maximum frequency	Cortex M55 MCU / 499.2 MHz	Qualcomm® Hexagon Processor v79 / 1651.20 MHz
Dedicated/shared	Dedicated for sensors	Shared with audio, other workloads
Hardware thread	1	4
RTOS	Zephyr v3.7.0 RTOS	Qualcomm Real Time (QuRT™) OS
TCM / Island memory	3.5 MB (2 MB ITCM + 1.5 MB DTCM)	2.5 MB Island memory
Sensor framework	Qualcomm® sensing hub (QSH) 4.0	QSH 4.0
Physical sensors	• Accelerometer / gyroscope • Magnetometer • Pressure • Ambient light sensor (ALS) • Photoplethysmogram (PPG) / Electrocardiogram (ECG) • Skin and ambient temperature	Electromyography / Electroneurography (EMG/ENG)
QSH contextual awareness	• QSH location • QSH BLE* • QSH geofence	• QSH Audio* • QSH WWAN • QSH Wi-Fi*
eNPU / AI - sensor data processing	Can access eNPU (using proxy driver on SWM, backend on LPAIDSP)	Can access eNPU (using Qualcomm® Neural Network (QNN) framework)
QTI QSH algorithms	• Absolute motion detection (AMD) • Significant motion detection (SMD) • Persistent Stationary / Motion Detection (PSMD) • Pedometer 3.0 • Tilt-to-wake • Activity Recognition on Wearables (ARW)* • Qualcomm Gyroscope Calibration (QGyroCal) • Qualcomm Magnetometer Calibration (QMagCal) • Rotation vector (RV) • Game rotation vector (GRV) • Geomagnetic rotation vector (GeoMagRV) • Gravity / Linear Acceleration	N.A.

Note:

QSH BLE, QSH Audio, QSH Wi-Fi, and ARW (marked with an asterisk (*)) are not supported in the current release. If needed, contact relevant QTI team for the support plan.

1.6.3 Embedded neural processing unit (eNPU)

eNPU module

Module	Capabilities
Version	eNPU6
Processing unit	Dual (Big/Little) eNPU 1024 + 256 MAC
Computational power	Up to 2 TOPS peak (int8)
Key features	• Offloads AI workload from SWM and LPAIDSP • Programmable through QNN • Doesn't require any application processor initiation or processing • Supports client priority and core selection/affinity
Typical use cases	• Big eNPU: Compute heavy workloads • Little eNPU: Memory bandwidth heavy workloads and lower complexity • Examples: Sensors, activity recognition, voice activation, noise suppression, and context detection.

See SW6100/SW6100P eNPU Guide — Linux Android Wear/Linux Wear, for more information.

1.6.4 Ambient wear microcontroller (AWM)

AWM subsystem

Module	Capabilities
CPU / Maximum frequency	Cortex M55 MCU / 499.2 MHz
RTOS	Zephyr v3.7.0 RTOS
TCM	2 MB (1.25 MB ITCM + 0.75 MB DTCM)
Physical sensors	RSB

1.7 Software features

This section describes the core software capabilities available on the platform, including the QSH framework, activity recognition, sensor data offload, rotary side button handling, and wrist/watch orientation support.

1.7.1 Qualcomm sensing hub framework

The following table describes key Qualcomm sensing hub (QSH) features and capabilities.

QSH features and capabilities

Feature	Capabilities
Framework execution model	Multi-threaded; can be updated to a single-threaded model using the Core QSH.
Performance	Event-driven parallel execution maximizes system performance.
Low-power operating mode	Low-power Island (LPI) mode with seamless transition to Normal mode.
Power management	Dynamic power management per use-case concurrency.
Memory management	Dynamic memory management support for sensors.

Feature	Capabilities
OS dependency	Portable across multiple platforms and operating systems.
Dynamic loading	Sensor can be dynamically loaded at runtime.
API compatibility	Supported through protobuf messages.
Multi-technology fusion platform	Supports Audio, Location, Bluetooth, Wi-Fi, and WWAN fusion.
New sensor/feature addition	Extensible without changes to the framework software.
Embedded AI	Integrated with the embedded neural processing unit (eNPU).
Sensor batching	Multi-player support optimizing system power and performance.
Remote file service	Enables persistent sensor configuration and calibration with dynamic updates.

1.7.2 Activity recognition

Activity recognition on wearables (ARW) 5.0 is an AI-powered algorithm that detects and classifies user activities in real time. It recognizes key movement patterns including biking, riding in a moving car, running or jogging, walking, and stationary or fidgeting behavior. ARW supports applications in fitness tracking, mobility analysis, and health monitoring.

The following table describes the supported user activity states.

User State	Description
BIKE	User on a non-stationary bike
CAR	User in a non-stationary car (Gasoline Car)
RUN	User running or jogging
STATIONARY	User stationary or fidgeting
WALK	User walking

1.7.3 Sensor data offload

The SW6100/SW6100P platform supports rendering sensor data on the display watchface. The implementation varies based on the software baseline, as described in the following table.

Software product	Modules involved
LW	• WHS • Display offload
LAW	• SSO • Display offload

1.7.4 Rotary side button

The rotary side button (RSB) sensor operates differently in Interactive and Ambient modes:

- Interactive mode: The application processor handles the RSB sensor.
- Ambient mode: The AWM handles RSB sensor, which is also called the RSB offload functionality.

RSB sensor handling is managed through:

- A shared/MUXed QUP interface between `SSC_QUP` and `TOP_QUP` over the enhanced general-purpose input/output (eGPIO).
- A common LDO for RSB, controlled by both the application processor and AWM.
- A Linux kernel driver for RSB in Interactive mode and a proprietary RSB driver for Ambient mode.
- A handshake mechanism for RSB offload entry and exit between the application processor and AWM.

See RSB use case, for a high-level depiction of RSB sensor handling in the QTI reference design.

1.7.5 Wrist/Watch orientation

This section provides an overview of wrist/watch orientation (WO) mode and the associated sensor functionality. The WO mode enables comfortable device viewing and operation for both right-handed and left-handed users. WO mode supports four configurations, based on the combination of wrist preference and crown sensor position.

The following table describes the orientation modes and their effect on crown position and screen rotation.

Wrist/watch orientation mode

Mode	Wrist	Crown	Screen rotation
0 (default)	Left	Right	–
1	Left	Left	180 °
2	Right	Right	–
3	Right	Left	180 °

Mode 0 is the default orientation. Change the orientation at any time using the **Settings** app. By default, the WO mode is enabled on both Wear OS (LW) and Android Wear OS (LAW). Each feature is supported through its respective HAL, service, and extended sensor daemon code implemented by QTI. The implementation handles Android interface definition language (AIDL) binder calls from the Google Clockwork framework.

See Wrist/Watch orientation, for more information about the design.

1.8 QTI reference design

This section provides sensors connectivity details of Qualcomm® SW6100/SW6100P wearables development platform (WDP) and wearables reference design (WRD) platforms.

> **Note:**
>
> QTI strongly recommends that the OEMs use bus interface GPIOs, interrupt/control GPIOs, and power rails, same as QTI reference design. If OEMs want to use different configuration, then review and receive approval from the QTI hardware, PMIC, and sensors teams in advance of designing a hardware platform.

1.8.1 Sensors on QTI reference design

Sensors on QTI reference platforms

Sensor type	Part (Vendor)	Interface
Accelerometer/gyroscope	LSM6DSV32X (ST)	I^3C
Magnetometer	AK09920C (AKM)	I^2C
Ambient light	OPT3004 (TI)	I^2C
Pressure	BMP585 (Bosch)	I^2C
Photoplethysmogram (PPG) / Electrocardiogram (ECG)	MAX86178 (ADI)	I^2C
Skin and ambient temperature	CT7117 (SensyLink) - 2 units	I^2C
Electromyography (EMG) / Electroneurography (ENG)	STENG1AX (ST) - 4 units	I^3C
Rotary side button (RSB) / Optical track sensor (OTS)	PAT9401E1 (PixArt)	I^2C

1.8.2 SSC Qualcomm® universal peripherals (QUP)

Note:

SSC QUP must be used for the sensor bus connectivity.

Supported SSC QUP

MSM GPIO#	SSC GPIO#	SSC QUP SE#	Supported protocols	MUXed with TOP QUP
GPIO_103-104	SSC_0-1	0	I^3C (/I^2C)	N.A.
GPIO_105-106	SSC_2-3	1	I^2C (/I^3C)	N.A.
GPIO_107-110	SSC_4-7	2	• UART-4W (/SPI/I^3C/I^2C) • SSC_SE2 support 6 lanes SSC_4-9 if needed	N.A.
GPIO_111-112	SSC_8-9	3	I^3C (/I^2C)	Yes, TOP_SE3
GPIO_113-116	SSC_10-13	4	2 I/Os + Debug 2W UART (/SPI/I^2C)	N.A.
GPIO_117-118	SSC_14-15	5	UART-2W (/I^2C)	Yes, TOP_SE6 (L2, L3)
GPIO_121-122	SSC_18-19	7	I^2C (/UART)	N.A.
GPIO_123-124	SSC_20-21	8	I^2C (/I^3C)	N.A.
GPIO_125-126, GPIO_119-120	SSC_22-23, SSC_16-17	9	• UART-4W (/I^3C/I^2C) • SSC_SE6 MUX'ed with SE9 GPIO_119-120	N.A.
GPIO_72 GPIO_40	SSC_24-25	10	I^2C	Yes, TOP_SE8
GPIO_131-134	SSC_26-29	11	SPI (/I^2C)	Yes, TOP_SE2

MSM GPIO#	SSC GPIO#	SSC QUP SE#	Supported protocols	MUXed with TOP QUP
GPIO_28-29	SSC_30-31	12	I^2C	Yes, TOP_SE9 (and TOP_SE6 L0, L1)
GPIO_127-130	SSC_32-35	13	SPI (/I^2C)	Yes, TOP_SE7

1.8.3 Sensor bus connectivity

SSC QUP usage for sensors

MSM GPIO#	SSC GPIO#	SSC QUP SE#	TOP QUP SE#	QUP Bus instance	Sensors connected
GPIO_103-104	SSC_0-1	0	N.A.	0	Accelerometer/ gyroscope
GPIO_105-106	SSC_2-3	1	N.A.	1	ENG-1, ENG-2
GPIO_111-112	SSC_8-9	3	N.A.	3	ENG-3, ENG-4
GPIO_121-122	SSC_18-19	7	N.A.	7	PPG/ECG
GPIO_123-124	SSC_20-21	8	N.A.	8	Magnetometer, pressure, ambient light, and skin and ambient temperature
GPIO_72 GPIO_ 40	SSC_24-25	10	N.A.	10	For RSB in Ambient mode
	N.A.	N.A.	8	N.A.	For RSB in Interactive mode

1.8.4 Sensor interrupt and control GPIOs

Sensors Control, Clock and Interrupt GPIOs

MSM GPIO#	Used/reserved on QTI reference platforms	Description
GPIO_12	ENG_PWM_OUT	ENG clock
GPIO_24	PPG_INT2	PPG sensor interrupt
GPIO_25	ALSP_INT_N	ALS interrupt
GPIO_26	ECG_FET_CTRL	Control ECG Electrode
GPIO_30	IMU_INT1	Accelerometer/gyroscope interrupt
GPIO_31	PPG_INT1	PPG sensor interrupt
GPIO_98	PRESS_INT	Pressure sensor interrupt*
GPIO_99	RSB_RESET_N	RSB hardware reset pin
GPIO_102	RSB_INT	RSB interrupt

Note:

GPIO_98 (marked with an asterisk (*)) isn't used by default in the SW6100/SW6100P QTI reference design but is reserved for the sensor interrupt purpose and can be utilized if required.

1.8.5 Sensor power rails

Power rails usage

Sensors power rails usage

Sensor type	Power rails
Accelerometer/gyroscope	Power rail: L13A (1.8 V) Usage: Connected to VDD/VDDIO of the sensor part
Magnetometer	
Ambient light	
Pressure	
Skin and ambient temperature	
EMG/ENG	
PPG/ECG	Power rail: L13A (1.8 V) Usage: Connected to VDD of the sensor part
	Power rail: L22A (4.5 V) Usage: Connected to VDD_LED of the sensor part
RSB	Power rail: L13A (1.8 V) Usage: Connected to VDD of the sensor part
	Power rail: VREG_BOB (3.6 V) Usage: Connected to VLD of the sensor part

Power rail and name mapping for sensor registry

Sensors power rails mapping for sensor registry

Power rail	Power rails name for sensor registry	
	SWM	LPAIDSP
L13A	/regulator/l13a	ldoa13
L22A	/regulator/l22a	N.A.

1.9 Use cases

This section provides high-level flow diagrams for various use cases.

1.9.1 Streaming use case

The following flow diagram depicts high-level overview of QSH sensor streaming by the application processor client.

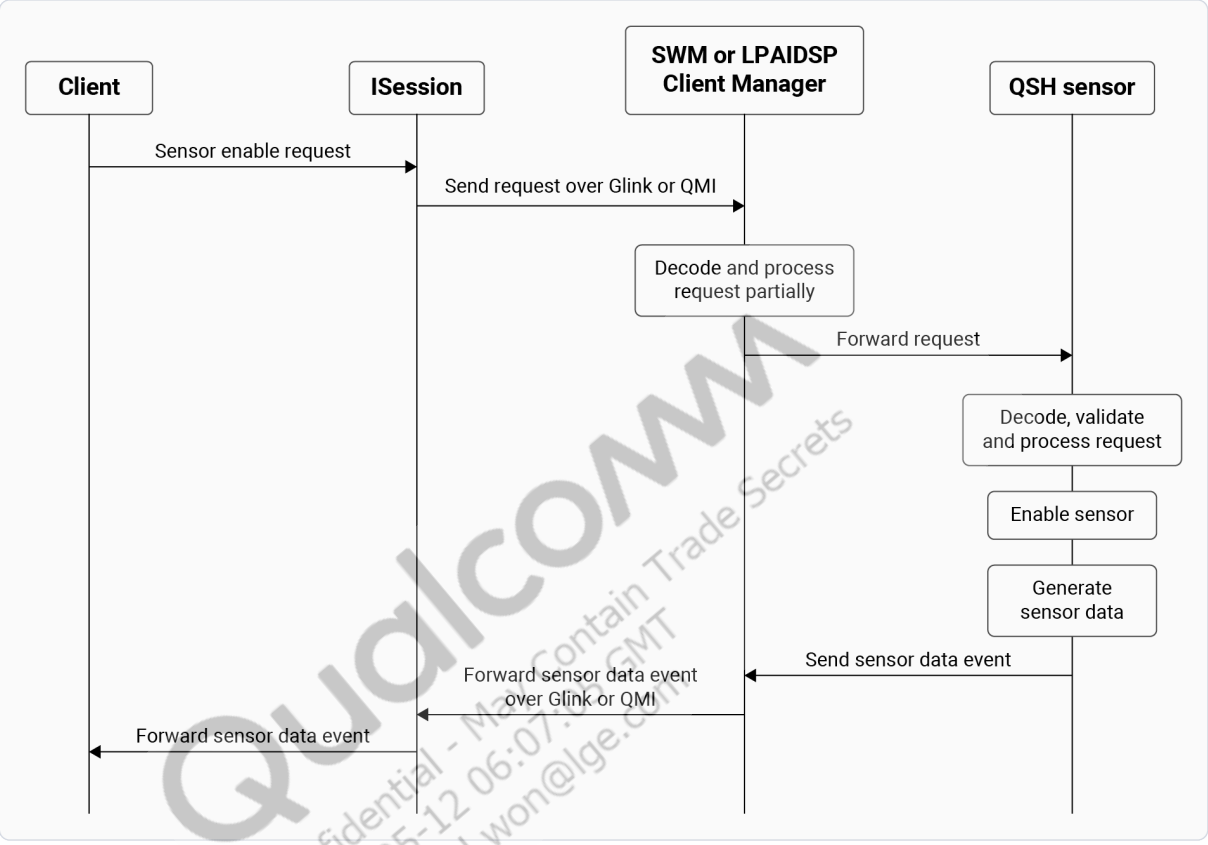


Figure: QSH sensor streaming

1.9.2 QSH router use case

The following flow diagram depicts high-level overview of the LPAIDSP sensor receiving SWM QSH sensor data using QSH router.

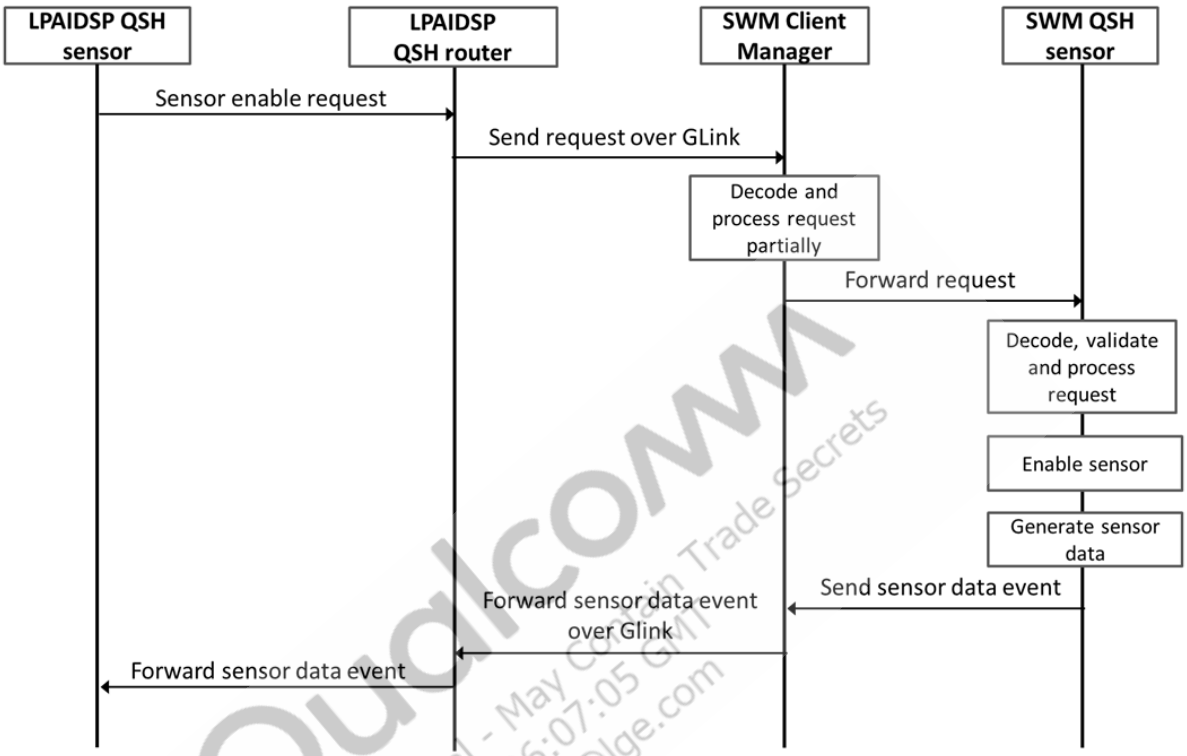


Figure: Receiving data using QSH router

1.9.3 eNPU sensors use case

The following flow diagram depicts a high-level overview of eNPU sensors use case with the LPAIDSP and SWM QSH.

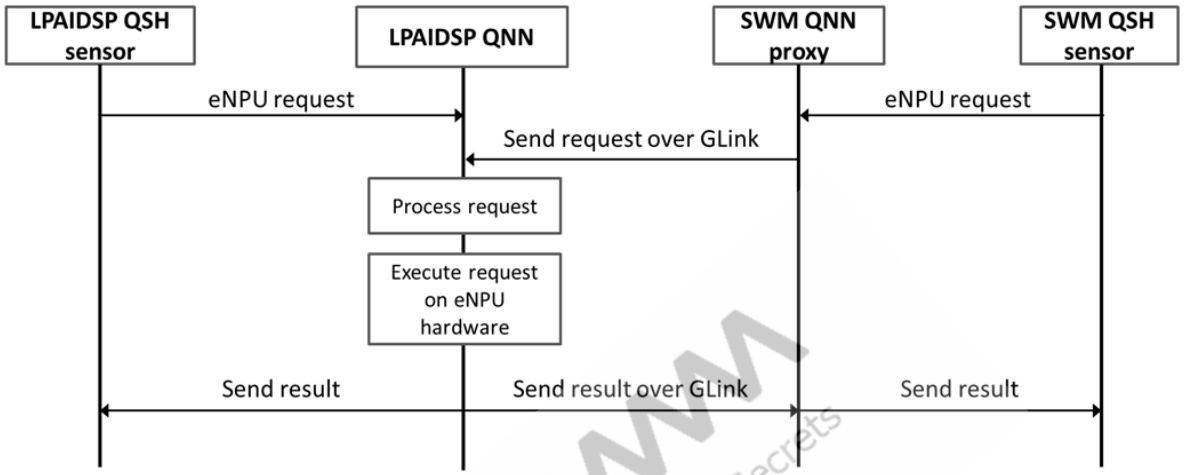


Figure: eNPU sensors use case with SWM and LPAIDSP QSH

1.9.4 RSB use case

The following flow diagram depicts high-level overview of RSB sensor handling in the QTI reference design.

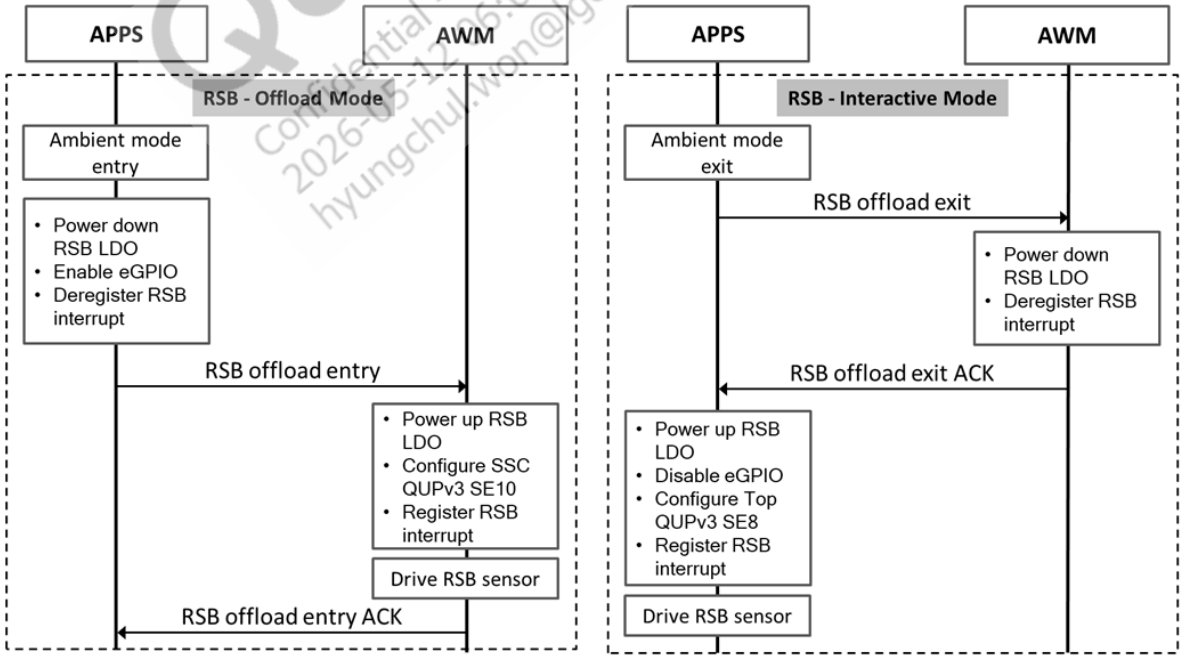


Figure: Handling RSB sensor in reference design

1.10 Wear health service

Google introduced wear health service (WHS) on the Wear OS platform to provide a uniform experience for health and fitness metrics. End-to-end WHS enablement involves multiple software modules across SW6100/SW6100P subsystems, with responsibilities split across Google, QTI, and OEM partners. Each party in the WHS ecosystem owns specific deliverables. Google defines the framework and test criteria, QTI provides the hardware abstraction layer (HAL) and reference implementation, and the OEM integrates required algorithms and customizations.

The following table describes the ownership roles and responsibilities for WHS enablement:

Ownership and the associated roles

Ownership	Role
Google	• WHS framework details and APIs. See Health Services on Wear OS, for more information about health services. • WHS framework specification, test details and test criteria: – Mandatory and optional algorithms for WHS. – Algorithm output event unit and format. – Test details and criteria for WHS use cases. • WHS test application, configuration file and application usage details.
QTI	• WHS Android interface definition language (AIDL) version 2.0 and version 3.0 APIs supported on Linux wearable (LW) 2.0 software products. • WHS HAL and health offload sensor in sensor wear microcontroller (SWM) to support WHS. • Native test application for WHS HAL level testing. • QTI pedometer algorithm enabled by default on SWM. • QTI pedometer algorithm doesn't provide the auto/pause/resume/start/stop, stats, and aggregation WHS features. • If required, OEMs can replace the QTI pedometer algorithm with a custom algorithm. See Integrate third-party provided algorithm or Develop algorithm, to integrate a custom algorithm. • Template WHS code provided as an example to work with the existing QTI pedometer algorithm.
OEM	• Receive WHS framework specification and test APK from Google. • If required, add other WHS mandatory and applicable optional algorithm(s), along with the QTI pedometer algorithm. • Customize the SWM-side QTI reference implementation as needed.

1.10.1 WHS health tracking

WHS defines three types of health tracking, each targeting different use cases for duration, interactivity, and automation.

- Exercise – Focused tracking of specific metric data for shorter duration than passive tracking. For example, an hour-long step count or heart rate tracking.
  - Supports pause and resume by the user.
  - Supports goal duration and goal threshold configuration by the user.
- Passive – Background tracking of metric data for longer durations. For example, step count or heart rate tracking for the whole day.
  - Supports goal duration and goal threshold configuration by the user.
  - WHS service triggers a reset to clear the daily stats.
- Auto-pause exercise – An extension to the *Exercise* tracking with automatic pause and resume capability. The metric algorithm detects and notifies the WHS service about device state, such as *rest* or *motion* device state.
  - Based on *rest* or *motion* events, WHS service triggers pause or resume for that specific algorithm.

To understand the health tracking requirements in detail, the OEM should obtain the WHS specification from Google.

> **Note:**
>
> QTI doesn't provide any reference algorithm or implementation for the *Auto-pause exercise* health tracking. The OEM can add or implement any required algorithm for *Auto-pause exercise* with respect to the WHS specification.

1.10.2 WHS functional overview

The WHS software stack processes health metrics through multiple layers, from the WHS service on the application processor through WHS HAL and down to QSH algorithms on SWM. The following figure shows the WHS functional overview architecture.

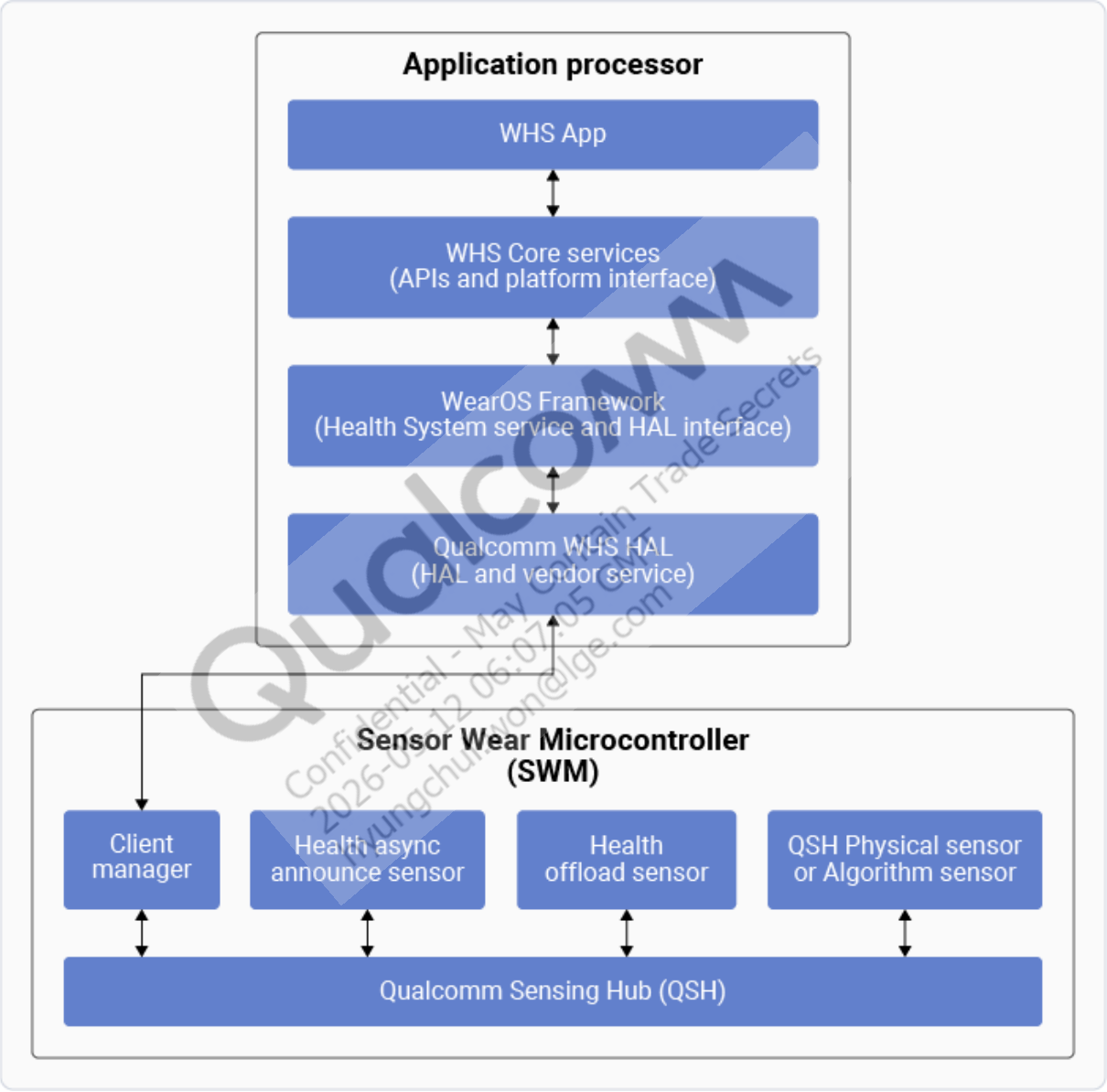
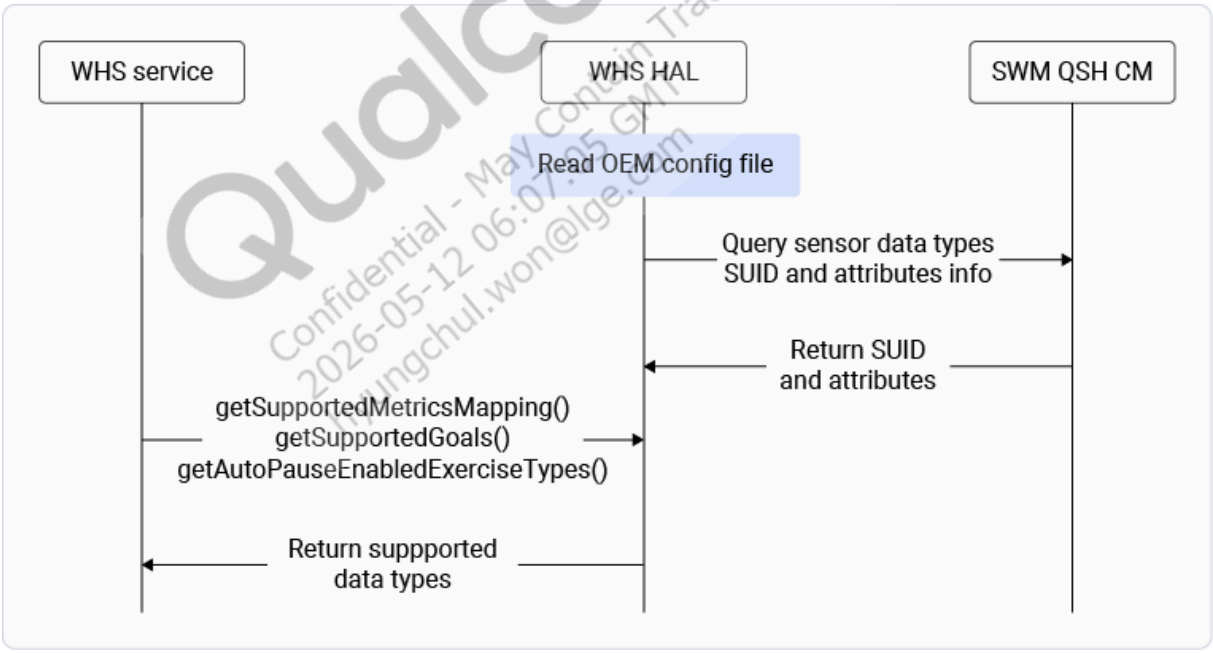


Figure: WHS functional overview architecture

- WHS HAL interface – Google defines the interface in `device/google/clockwork/interfaces/healthservices/`. QTI's WHS HAL creates associated header files, for WHS HAL functionality, at `vendor/qcom/proprietary/vienna/whs-sensors/whs-hal-2.0/framework/inc`. The interface includes data types, exercise types, request types, output data types, associated data structures, and so on.

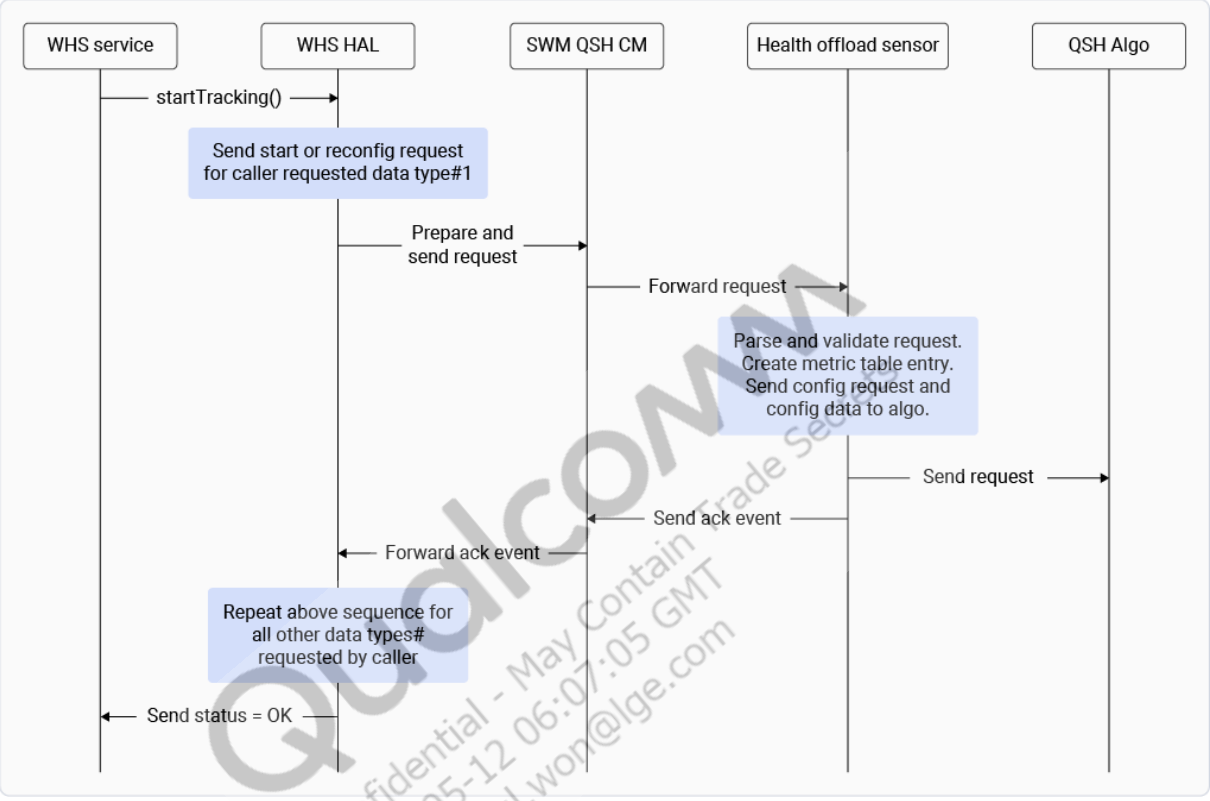
- WHS core services – Provides the APIs for WHS application. The health system service interacts with WHS HAL using WHS HAL interface APIs.
- WHS HAL initialization – Uses the `/vendor/etc/sensors/oem_config.xml` file on the device to start the sensor unique identifier (SUID) lookup for the specified sensor types. In the application processor source tree, this file is located at `vendor/qcom/proprietary/vienna/whs-sensors/whs-hal-2.0/config/oem_config.xml`. It defines all supported WHS HAL data types and capabilities. OEMs must update this file to include the required WHS algorithm sensor type(s) and any additional capabilities.
- Sensor discovery – WHS HAL performs a lookup operation on every `sensor_type` field in `oem_config.xml`. The available sensors list is stored in `/mnt/vendor/persist/sensors/whs_sensors_list.txt` on the device. This file is created on the first boot after the flashing software images and in absence of file. This file is used as reference on the future boots. During sensor discovery, WHS HAL waits for the shorter duration of: all listed sensors becoming available, or the maximum timeout being reached.
- Supported queries – The list of available metric(s), goal(s), and auto-pause-supported exercise(s) is returned to the WHS service layer when the respective API triggers. For example, `getSupportedMetricsMapping()`, `getSupportedGoals()`, and `getAutoPauseEnabledExerciseTypes()`.



- Two QSH sensors are implemented in the SWM QSH for WHS functionalities:
  - Health offload sensor – It's a primary QSH sensor for WHS. It parses and validates requests from WHS HAL, manages metric metadata, forwards request to the respective algorithm, handles algorithm data event, and so on. See `sns_health.proto`, for more information.

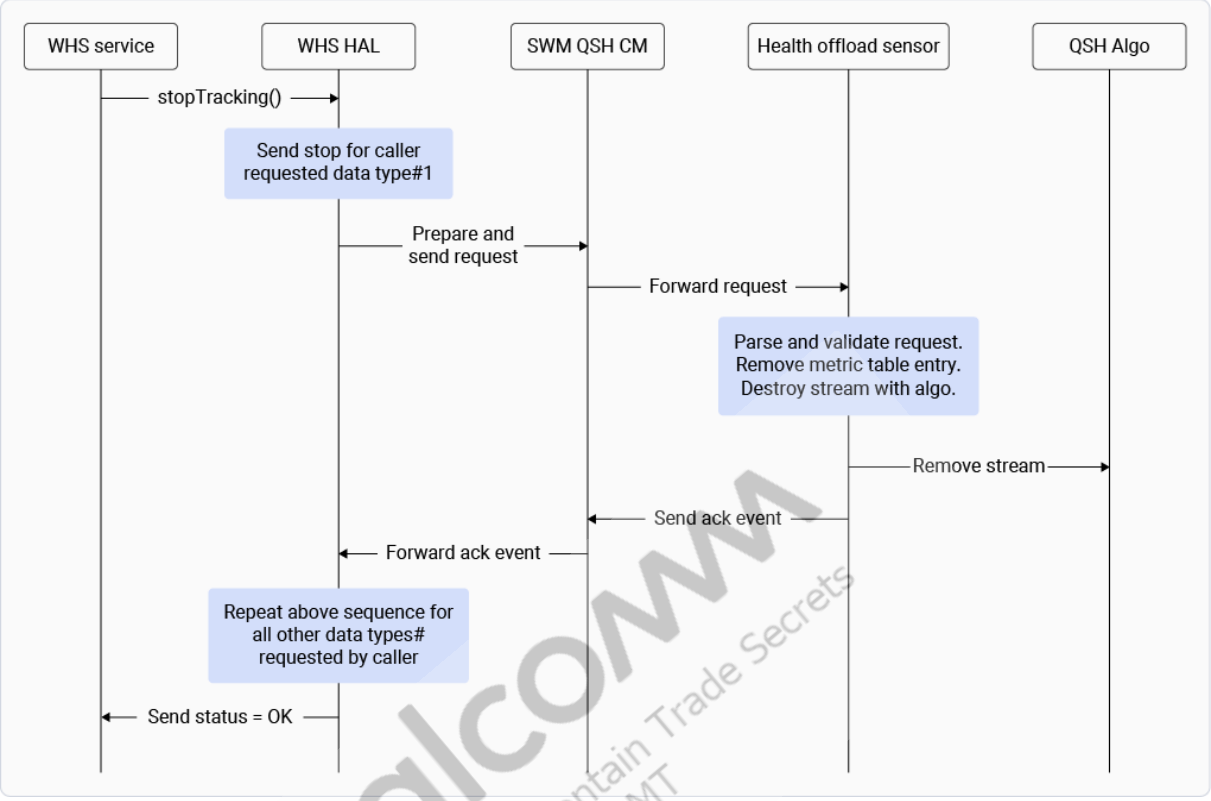
– Health async announce sensor – It's a QSH sensor to send goal event and location status updates to the WHS HAL as asynchronous updates without batching.

- After sensor type discovery, the WHS service can start tracking an exercise or passive metric at any time. The following figure shows the start tracking call flow:



1. The WHS service calls `startTracking()` API to start a metric. This creates the respective algorithm instance in the SWM.
2. WHS service asynchronously receives an acknowledgement event from the health offload sensor on SWM with the result of `startTracking()` request processing.
3. The tracking configuration can be updated later using `updateTrackingConfig()` API. The start tracking configuration includes sample rate, report period (batch period), and GPS enable flag.

- After a metric starts successfully, the WHS service can stop it using `stopTracking()` API. The following figure shows the stop tracking call flow:

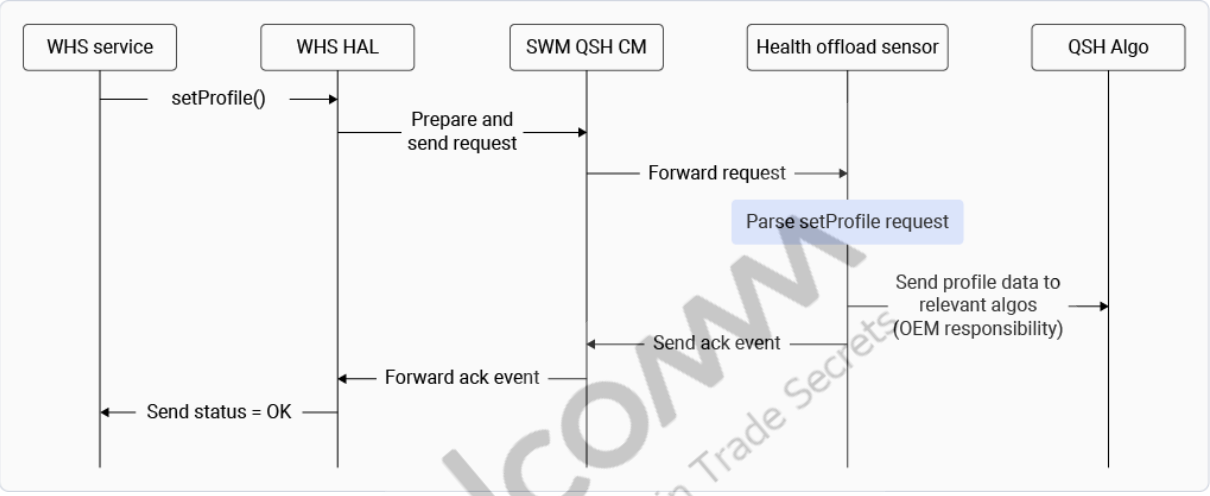


The QSH algorithm associated with the metric receives a remove request. WHS service asynchronously receives an acknowledgement event from the health offload sensor with the SWM containing the result of `stopTracking()` request processing.

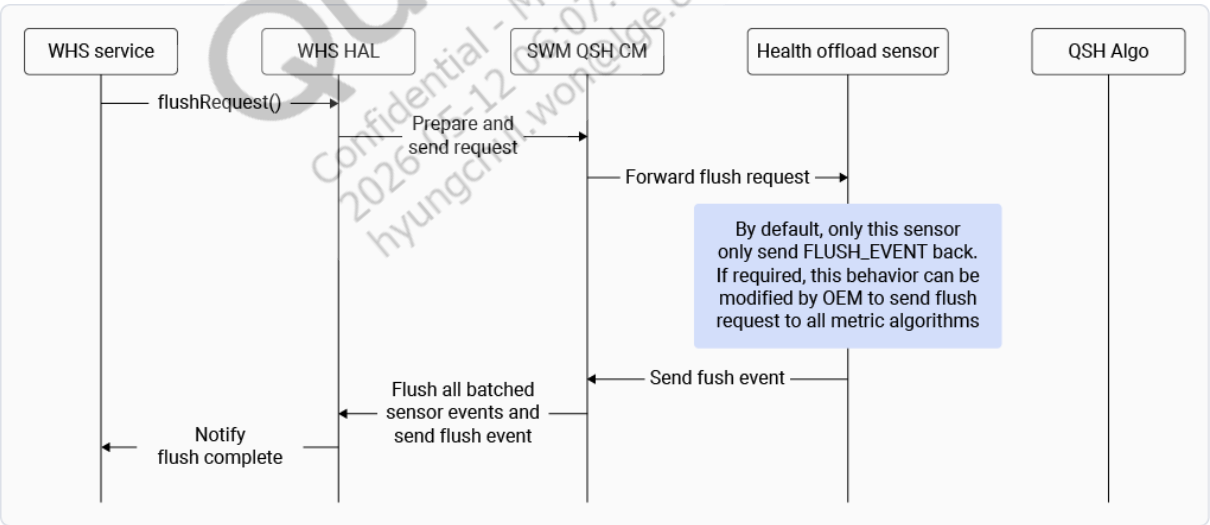
- WHS service can pause a metric using `pauseTracking()` API and resume it using `resumeTracking()` API. WHS service asynchronously receives acknowledgement events from the health offload sensor on SWM containing the result of `pauseTracking()` and `resumeTracking()` request processing.
  - QTI's reference code handles the pause request by removing the stream for the associated metric algorithm sensor. The OEM can choose to handle it differently without removing the stream in health offload sensor and associated metric algorithm.
  - QTI's reference code handles the resume request by starting the stream for the associated metric algorithm sensor (since pause stopped it). The OEM can choose to handle it differently in health offload sensor and associated metric algorithm.
- All events generated by a metric algorithm are of wake-up type for the SW6100/SW6100P application processor. These samples wake up the application processor from suspend state to deliver samples to WHS HAL.
- WHS HAL acquires a wake lock on event receipt and releases it after propagating data to WHS service. The client application can acquire and release the wake lock, if required.
- The event received by WHS HAL is a byte array. To decode the event to the intended format, WHS HAL uses a predefined hard-coded mapping of the metric data type. This mapping is in

the `datatype_output_format()` function defined in `vendor/qcom/proprietary/vienna/whs-sensors/whs-hal-2.0/framework/inc/whs_types.h`.

- Whenever the user changes profile details, the WHS service sends a set profile request using `setProfile()` API. WHS HAL forwards the request with the payload to the health offload sensor. The OEM implements the code to broadcast profile data to the corresponding algorithms. The following figure shows the set profile call flow:

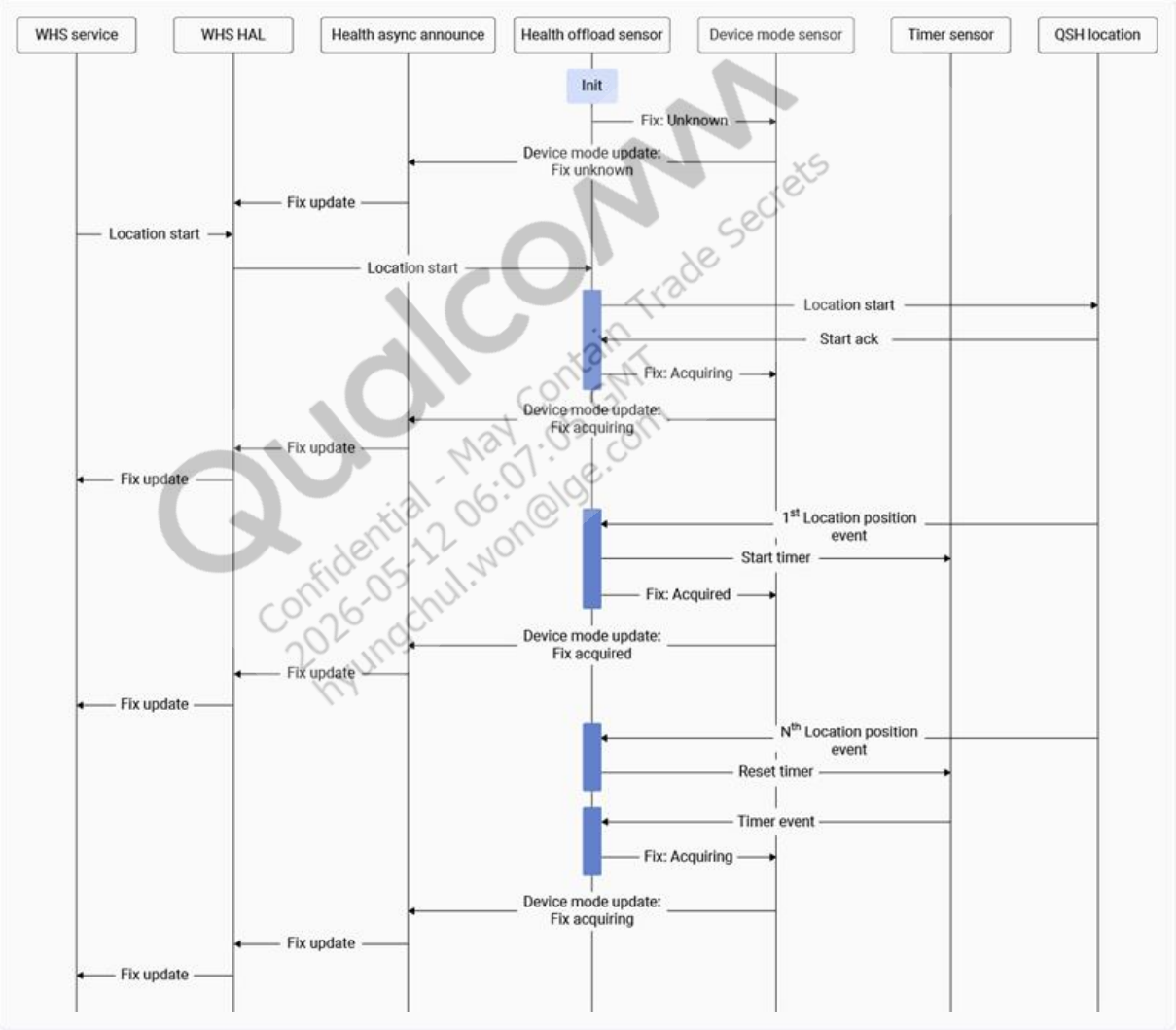


- The `flush()` API flushes batched or available algorithm data that's delivered to the WHS service.



- Set goals using `addGoal()` and remove them using `removeGoalById()` API.
- When WHS service requests for the GPS data, the health offload sensor receives the request, enables QSH-Location sensor, and returns the GPS data back to the WHS service.
  - By default, the GPS state is set to *unknown* in health offload sensor.

– After the QSH-Location starts and the health offload sensor gets the ACK event from QSH-Location then the GPS state sets to *Acquiring* and a timer starts with 3 seconds duration.
– If the health offload sensor receives position event within 3 seconds then the timer stops and the GPS state moves to *Acquired*. But if the health offload sensor doesn't get position event, then upon timer event handling and GPS state stays in *Acquiring* state. In such timeout case, if the position event is received after timeout, then the GPS state moves to *Acquired* state.
– If required, other WHS metric algorithms in the SWM QSH can leverage the GPS status directly from the QSH location sensor.



1.10.3 Develop

See Wear health service, for information about developing WHS.

1.11 Sports sensor offload

Sports sensor offload (SSO) provides session-based sensor and algorithm enablement with the capability to render metric data on the display watchface. The sensor metric data path and display offload path are isolated and can be used independently based on the use case.

Sensor metric data path

SSO uses the `Sidekickmetric` functionality to start required metric algorithms. The `Sidekickmetric` HAL sends the request to SWM QSH. Depending on the metric request configuration, metric data is sent back to the SSO client through `Sidekickmetric` HAL.

Display offload path

- SSO uses the `UxOffload` functionality to offload the display assets for rendering and triggers the QSH Display Bridge to act as a client to SWM QSH for enabling the desired metric algorithm.
- The rendering of metric data over display is handled by `UxOffload` like any other offload use case. In Ambient mode, the AWM renders the data on the watchface. In Interactive mode, the application processor renders data on the watchface.

1.11.1 SSO functional overview

The SSO architecture spans the high-level OS (HLOS) layer (Sidekickmetric HAL, UxOffload) and the SWM layer (formatter sensor, QSH algorithms). The following figure shows the SSO functional overview architecture.

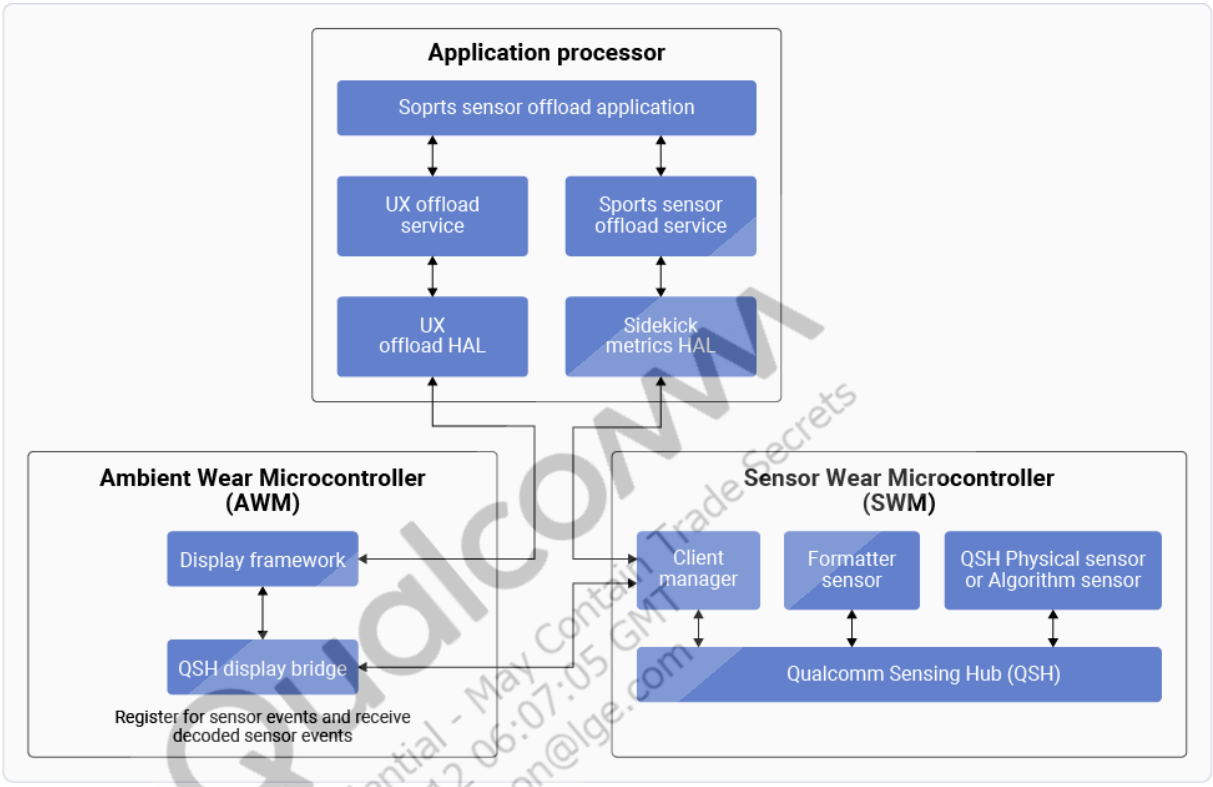


Figure: SSO functional overview architecture

Sensor metric data path

- `Sidekickmetric` HAL initialization – Uses the `vendor/etc/sensors/metric_sensors_list.txt` file on the device to start the sensor unique identifier (SUID) lookup for the specified sensor types. In the application processor source tree, this file is located at `device/qcom/common/ssohal/metric_sensors_list.txt`. OEMs must update this file to include the required metric algorithm sensor types.
- Metric instance lifecycle – The client gets a list of available metric sensors, creates metric instances for a particular algorithm, and starts the metric instance and algorithm. The client can start respective metric algorithm only after a metric instance is created. The client can stop and restart a metric instance, such as a session, after the required interval. When stopped, the metric algorithm doesn't generate output. A metric instance can be destroyed after the session ends.

- `recordData` parameter – When set to `TRUE` in the metric request, the metric request is routed through `Sidekickmetric` HAL to the SWM formatter sensor. After the formatter

sensor processes the metric request, it forwards the request to the required algorithm on SWM. The HLOS client application then receives the metric algorithm output.

- `Offload` parameter – Ignore this parameter in the metric request on SW6100/SW6100P.
- Batch groups – `Sidekickmetric` HAL supports two batch groups: *Batching* and *Streaming*. The SSO client should use the *Batching* group to avoid frequent HLOS wake-ups using the `batch()` API. A single *Batching* group in `Sidekickmetric` HAL allows optimal memory use in SWM.
- Formatter sensor – `Sidekickmetric` HAL sends the request to formatter sensor in the SWM QSH. The formatter sensor includes a reference implementation that supports step count, calorie, and distance metrics based on the QTI pedometer algorithm. OEMs should use this implementation as a baseline and add support for any new algorithms as needed. The formatter sensor performs the following key activities:
  - Process client metric request and create metric instance.
  - Create stream and activate algorithms using respective `protobuf` files.
  - Convert received data from algorithms.
  - Generate metric output and notify the client.
- Wake-up behavior – All samples generated by a metric algorithm are of wake-up type for the SW6100/SW6100P application processor. If the application processor is suspended, then the samples wake up the application processor to deliver the metric data to `Sidekickmetric` HAL. After receiving data from SWM into `Sidekickmetric` HAL, the HAL doesn't acquire a wake lock. The client application must acquire and release the wake locks.

Display offload path

- The SSO client uses `UxOffload` APIs to offload the required display assets to AWM.
- If a display asset maps to an existing metric algorithm, then the QSH Display Bridge acts as a client to SWM QSH and sends a request for the formatter sensor.
- The formatter sensor performs the same activities as described in the SSO sensor metric data path section.
- After QSH Display Bridge receives the metric data, it renders on the display watchface.

1.11.2 Subsystem restart handling

Subsystem restart (SSR) handling has the following behavior:

- `Sidekickmetric` HAL always stores the latest event received for each active metric ID.
- When the SWM/LPAIDSP SSR occurs:
  - `Sidekickmetric` HAL closes all Glink connections and re-establishes them after the SWM/LPAIDSP is up.
  - The application processor client receives subsystem down and up notifications.
  - Once the subsystem is up, the SSO client can resend all or required metric requests to SWM.

- `Sidekickmetric` HAL passes the stored latest value (before subsystem restart) to the formatter sensor as a start offset/value when each metric restarts.
- QTI algorithms don't support start offset/value, but OEM algorithms can use the start value, if required.
- In the existing formatter reference implementation for the QTI pedometer algorithm, the start offset/value provided directly by the SSO client or by `Sidekickmetric` HAL after SSR, is stored to calculate the future step count as, `start offset/value + current step count`.

1.11.3 Develop

See Sports sensor offload, for information about developing SSO.

1.12 Wrist/Watch orientation

The wrist/watch Orientation (WO) mode enables comfortable device viewing and operation for both right-handed and left-handed users. The UI rotation and rendering is handled by the display framework and associated modules and subsystems. This section focuses on the impact and handling of orientation mode changes by QSH modules.

WO mode changes require orientation-sensitive QSH sensors to adapt to new orientation mode. A dedicated Device Mode sensor notifies QSH sensors of orientation mode changes.

The following figures show the WO mode setting menu on the LW and LAW devices when mode 0 (Wrist = Left, Crown = Right) is selected.

Figure: WO mode setting for LW

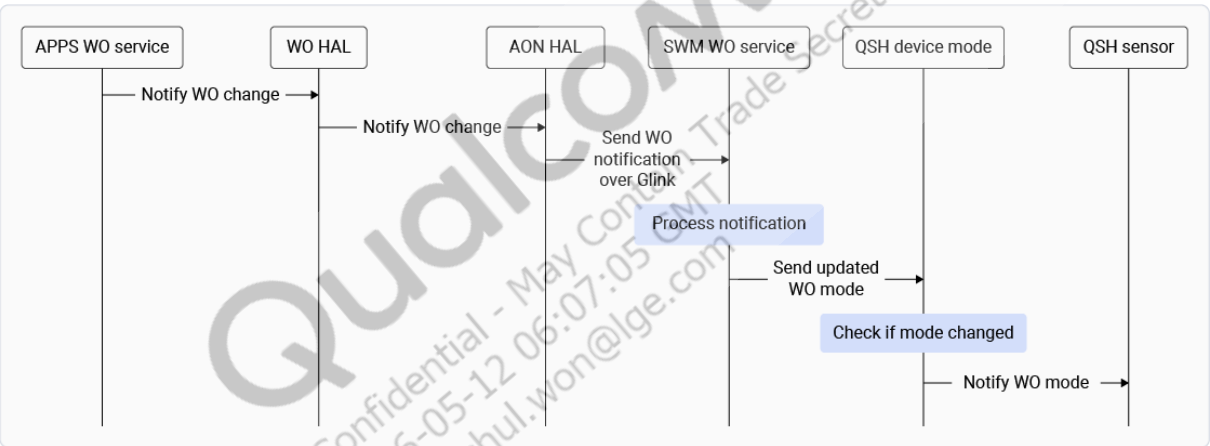
Figure: WO mode setting for LAW

1.12.1 Functional overview

This section provides a high-level functional overview of how WO mode change is notified to the QSH sensor.

- On the application processor, the WO service and HAL identify and notify the orientation mode. A wrist orientation service starts in the SWM to receive notifications.
- During boot and whenever orientation mode changes, the application processor notifies the current orientation mode to the SWM wrist orientation service over Glink.
- The SWM wrist orientation service processes the orientation mode notification and sends an update to the QSH Device Mode sensor.
- If the received orientation mode differs from the currently known mode, the Device Mode sensor generates a notification event for its clients.

The following flow diagram shows the orientation mode update process from application processor till QSH sensor.



1.12.2 Impact analysis

The following table describes the impact of WO mode changes on each module:

Module	Description
Physical sensor driver	• Sensor data is always in the Android coordinate reference, regardless of wrist mode. • Sensor data from drivers must always follow the Android coordinate system.
QTI calibration algorithm sensors	• Both magnetometer and gyroscope calibration algorithms aren't sensitive to the WO.
QTI algorithm sensors	• Only the Wrist Tilt algorithm, from QTI deliverables, is sensitive to the orientation mode. It's improved to work correctly in all orientation modes.
OEM-integrated QSH sensors	• Only orientation-sensitive sensors must adapt to the provided guidelines to work correctly in all orientation modes.
Application processor clients	• Not impacted, since clients always receive sensor data in the Android coordinate system.
Factory calibration	• No changes to factory calibration are required when switching the WO mode. Bias is a sensor characteristic and doesn't depend on software settings. Factory calibration can be performed in any WO mode. • In the event of a factory data reset, removing the `/data` partition resets the WO mode, and subsequent sensor data defaults to the initial mode.

1.12.3 Device Mode sensor

The Device Mode sensor on SW6100/SW6100P supports orientation mode update notifications. Qualcomm sensing hub (QSH) sensors register for these notifications to stay synchronized with wrist orientation changes.

The Device Mode sensor performs the following activities:

- Initializes with `SNS_DEVICE_STATE_UNKNOWN` orientation mode until the current orientation mode is received from the application processor.
- Once the Device Mode sensor receives an orientation mode update, all existing clients receive the notification event.
- As Device Mode is an on-change sensor, any new client receives the current orientation mode after the client request is processed successfully.

The following table describes the orientation modes and the notification events generated by the Device Mode sensor.

Mode	Wrist	Crown	Screen rotation	sns_device_mode.proto	
				SNS_DEVICE_MODE_WEAR_WRIST_RIGHT (9)	SNS_DEVICE_MODE_WEAR_DISPLAY_ROTATED (10)
0 (default)	Left	Right	–	INACTIVE	INACTIVE
1	Left	Left	180°	INACTIVE	ACTIVE
2	Right	Right	–	ACTIVE	INACTIVE
3	Right	Left	180°	ACTIVE	ACTIVE

1.12.4 Adapting QSH sensor to orientation change

Only the orientation-sensitive QSH sensors need to follow the guidelines for adapting to orientation mode changes. See Wrist/Watch orientation, for information about developing WO.

2 Bring up sensors

The sensors bring up and customization typically comprises the following activities:

- QSH drivers and algorithms customization: Based on the customization requirements, see the following applicable topics:
  - Customize QTI reference design sensor driver
  - Integrate third-party sensor driver
  - Integrate vendor provided algorithm
  - Develop algorithms
- Sensor axis mapping
  - See Sensor axis mapping and calibration, for more information.
- Sensor calibration
  - Prerequisites
    - Bring up of all physical sensors is complete.
    - Axis mapping is correct for all the sensors.
  - See Sensor axis mapping and calibration, for more information.
- Sensor configuration and registry functionality
  - See Sensors Execution Environment (SEE) - Sensor Config And Registry (English), for the overview video training.
  - See the *Registry* section in Sensors Execution Environment (SEE) Sensors Deep Dive, for documentation.
- Sensor configuration and registry JSON paths – `hub1` is for LPAIDSP and `hub2` is for SWM:
  - On device

JSON configuration directory	`/vendor/etc/sensors/hub1/config` and `/vendor/etc/sensors/hub2/config`
Sensor registry JSON directory	`/mnt/vendor/persist/sensors/hub1/sensors/registry/registry` and `/mnt/vendor/persist/sensors/hub2/sensors/registry/registry`

  - Application processor source code

JSON configuration directory	<hlos_workspace>/vendor/qcom/proprietary/sensors-ship/config/registry/hub1/sensors/vienna and <hlos_workspace>/vendor/qcom/proprietary/sensors-ship/config/registry/hub2/sensors/vienna.

- SWM source code
  - Sensor drivers – wm_proc/sensinghub/sensors/drivers
  - Algorithm – wm_proc/sensinghub/sensors/algorithms
  - POR make file – wm_proc/sensinghub/platform/chipset/aspen/see/por.cmake
- LPAIDSP source code
  - Sensor drivers – adsp_proc/ssc_drivers
  - Algorithm – adsp_proc/qsh_algorithms
  - POR make file – adsp_proc/qsh_platform/chipset/aspen/qsh/por.py

2.1 SWM QSH sensor integration

This section covers customization steps for SWM.

2.1.1 Customize QTI reference design sensor driver configuration

This section is applicable only if the sensor part used is the same as the SW6100/SW6100P QTI reference design but the sensor GPIOs, interrupt pin, or bus type used are different. See the *Registry in SEE* section of Sensors Execution Environment (SEE) Sensors Deep Dive while performing the following steps.

SWM side changes

- If the GPIOs or bus interface type used for sensor is different from the QTI reference design sensor, then perform the following to configure the QUP instance as required.
  - See SW6100/SW6100P Device Interfaces Guide to check whether the intended bus interface type is supported by the SSC QUP SE# serial engine. If required, modify the intended QUP instance and serial engine configuration.

> **Note:**
>
> For any QUP instance and serial engine configuration support, raise a Salesforce case with Buses team with problem area as *BSP/HLOS, Driver/Peripheral, Peripheral Buses (UART/I2C/SPI/TSIF/etc) - non AP*.

- If the bus type for SSC QUP SE# 0 is changed from default I^3C to any other bus type, then comment out CONFIG_QC_SNS_I3C_DEVICE_INIT_PRIO macro in the wm_proc/sensinghub/config/prj_aspen_lpai_swm_board.conf file.

Application processor side changes

- Modify the sensor driver's JSON configuration.
  - Use the following table to locate the QTI reference design sensor's JSON configuration file under `/vendor/etc/sensors/hub2/config/` directory and pull the file from the device.

Sensor type	Part (Vendor)	JSON configuration file
Accelerometer/gyroscope	LSM6DSV32X (ST)	`aspen_lsm6dsv_0.json`, `aspen_wdp_lsm6dsv_0.json`, and `aspen_wrd_lsm6dsv_0.json`.
Magnetometer	AK09920C (AKM)	`aspen_ak991x_0.json`, `aspen_wdp_ak991x_0.json`, and `aspen_wrd_ak991x_0.json`.
Ambient light	OPT3004 (TI)	`aspen_opt3001_als_0.json`.
Pressure	BMP585 (Bosch)	`aspen_bmp5_0.json`.
PPG/ECG	MAX86178 (ADI)	`aspen_max86178_0.json` and `aspen_max86178_reg_0.json`.
Skin and ambient temperature	CT7117 (SensyLink) - 2 units	`aspen_ct7117x_0.json` and `aspen_ct7117x_1.json`.

  - Use the QUP instance ID as `bus_instance` registry item value of the driver.
  - Set `bus_type` item value to match the bus interface type.
  - If the sensor uses SPI or I²C interface, then set `dri_irq_num` to the GPIO number used for the interrupt.
  - Set `slave_config` item value with the *Slave Address* for I²C/I³C bus interface or *Chip Select Number* for the SPI bus interface.
  - Set `min_bus_speed_khz` to the minimum bus speed supported by the sensor. Follow similar steps as described for setting the `max_bus_speed_khz` item.
  - Set `max_bus_speed_khz` to the maximum bus speed supported as per the sensor Data Sheet and the maximum speed supported by bus interface of the QUP serial engine.

- Push the modified configuration JSON files to the device `/vendor/etc/sensors/hub2/config/`.

- Remove all the existing sensor driver registry files from the device. To perform this step, locate the file name prefix in the driver JSON configuration file as mentioned below and then remove all the registry files with the same prefix under `/mnt/vendor/persist/sensors/hub2/sensors/registry/registry/` directory.

```
"config": {
  ...
```

```
},
"prefix": {
```

- Reboot the device.
- After reboot, verify that the updated driver registry files are present in `/mnt/vendor/persist/sensors/hub2/sensors/registry/registry/` directory.

Verify sensor

See Tools to verify sensors.

2.1.2 Integrate third-party sensor driver

Perform the following steps to integrate third-party sensor drivers on the SWM. See the *Registry in SEE* section of Sensors Execution Environment (SEE) Sensors Deep Dive while performing the steps.

SWM side changes

1. Acquire the directory from vendor, which contains the driver, associated SCons build file, JSON configuration files for driver, and any custom protobuf file for the driver.
2. Place the acquired directory in `wm_proc/sensinghub/sensors/drivers`.
3. If the driver has custom `.proto` protobuf file, copy the file to `wm_proc/sensinghub/api/pb` directory.
4. Enable the sensor driver for compilation.
   a. See Convert SCons to CMake, to create `CMakeLists.txt` file under the build directory for compiling.
   b. See Disable SWM QTI reference design sensor, if the new sensor driver must replace an existing QTI reference design driver of the same sensor type.
   c. In the driver's `CMakeLists.txt` file look for the name used as target under all the occurrences of `add_ssc_su()` function. Add name(s) into `include_sensor_vendor_libs` array in the `wm_proc/sensinghub/platform/chipset/aspen/see/por.cmake` file.
   d. Include the driver directory to compile using the `add_subdirectory()` function in the `wm_proc/sensinghub/sensors/drivers/cmake_build/ssc_drivers.cmake` file.
5. If the bus interface GPIOs or bus interface type used for a sensor is different from the QTI reference design sensor, then perform the following steps to configure the QUP instance as required.
   a. See SW6100/SW6100P Device Interfaces Guide to check whether the desired bus interface type is supported by the `SSC QUP SE#` serial engine. If required, modify the required QUP instance and serial engine configuration.

> **Note:**
>
> For any QUP instance and serial engine configuration support, raise a Salesforce case with the Buses team with problem area as *BSP/HLOS, Driver/Peripheral, Peripheral Buses(UART/I2C/SPI/TSIF/etc) - non AP*.

b. If the bus type being used `SSC QUP SE#0` as I^3C, then replace the `lsm6dso0` keyword with a suitable keyword for the third-party driver in the following locations:

- `wm_proc/sensinghub/platform/chipset/aspen/see/sensinghub.overlay` file and
- `SENSOR_DEVICE_DT_DEFINE(DT_NODELABEL(lsm6dso0),...` line in the `wm_proc/sensinghub/platform/pal/i3c/zephyr/sns_com_port_i3c.c` file.

c. If the bus type for `SSC QUP SE#0` is changed from default I^3C to any other bus type, then comment out `CONFIG_QC_SNS_I3C_DEVICE_INIT_PRIO` macro in the `wm_proc/sensinghub/config/prj_aspen_lpai_swm_board.conf` file.

6. After compiling the SWM code, verify that the driver registration function (specified by `register_func_name` in the driver SCons file) is added to `sns_static_drivers_list[]` in the `wm_proc/zephyr/build/lpai_swm_img/arm/aspen.swm.prod/modules/sensinghub/sensors/drivers/sns_static_drivers.c` file.

Application processor side changes

1. If the sensor driver is for a new sensor type that's different than the QTI reference design sensor types and if the sensor needs to be exposed to the sensor HAL, Android framework, and Android app then:

   a. Check if the sensor HAL source code has associated `cpp` file of the required new sensor type at `<hlos_workspace>/vendor/qcom/proprietary/sensors-android/sensors-hal-2.X/sensors`.

   b. If the `cpp` file for the required new sensor type doesn't exist, then check with the vendor to get the file. If the vendor doesn't have such file, then see the existing `electro_neuro_graph.cpp` or `ambient_light.cpp` file and create a new `cpp` file to support the new sensor type.

2. If the driver has custom `.proto` protobuf file, then:

   a. Copy the file to `<hlos_workspace>/vendor/qcom/proprietary/sensors-ship/api/proto` directory.

   b. Push it to the `/vendor/etc/sensors/proto/` device directory.

3. Update the driver registry files received from the sensor vendor.

   a. Check if the configuration section at start of each driver configuration JSON file contains the expected values for `hw_platform` and `platform_family` (if present) items. The values must match the following command output. If the value from the command output isn't present in the respective item, then add the value to the respective item:

```
adb shell cat /sys/devices/soc0/hw_platform
adb shell cat /sys/devices/soc0/chip_family
```

b. Use the QUP instance ID as `bus_instance` registry item value of the driver.

c. Set `bus_type` item value to match the bus interface type.

d. If the sensor uses a SPI or I^{2}C interface, then set `dri_irq_num` to the GPIO number used for the interrupt.

e. Set `slave_config` item value to *Slave Address* for I^{2}C/I^{3}C bus interface or *Chip Select Number* for SPI bus interface.

f. Set `min_bus_speed_khz` to the minimum bus speed supported by the sensor. Follow similar steps as described to set the `max_bus_speed_khz` item.

g. Set `max_bus_speed_khz` to the maximum supported bus speed as per the sensor Data Sheet and the maximum speed supported by the bus interface of the QUP serial engine.

   i. To know the QUP serial engine's maximum frequency for bus interface type, see SW6100/SW6100P Device Interfaces Guide.

   ii. To know the sensor hardware maximum bus interface frequency support, see the sensor Data Sheet.

   The following table lists the maximum supported frequency for sample QUP serial engines. (This is only for reference. See the above mentioned documents to confirm the values.)

Bus interface	Maximum frequency supported
I^{3}C	12.5 MHz
I^{2}C	10 MHz
SPI	50 MHz

4. Add the JSON file name into `/vendor/etc/sensors/hub2/config/json.lst`.
5. Push the modified configuration JSON files to the `/vendor/etc/sensors/hub2/config/` device directory.
6. Reboot the device.
7. After reboot, verify that the updated driver registry files are present in `/mnt/vendor/persist/sensors/hub2/sensors/registry/registry/`.

Verify sensor

See Tools to verify sensors.

2.1.3 Integrate third-party provided algorithm

To integrate the third-party sensor algorithm on the SWM:

SWM side changes

1. Acquire the directory from vendor, which contains the algorithms, associated SCons build file, JSON configuration files, and any custom protobuf file for the algorithm.
2. Place the acquired directory in `wm_proc/sensinghub/sensors/algorithms`.
3. If the driver has custom `.proto` protobuf file, copy the file to `wm_proc/sensinghub/api/pb`.
4. Enable the sensor algorithm for compilation.
    a. See Convert SCons to CMake, to create `CMakeLists.txt` file under the build directory for compiling.
    b. If the new sensor algorithm must replace an existing QTI reference design algorithm of the same sensor type, then follow Disable SWM QTI reference design sensor.
    c. In the algorithm's `CMakeLists.txt` file look for the name used as target under all the occurrences of `add_ssc_su()` function. Add name(s) into `include_sensor_vendor_libs` array in the `wm_proc/sensinghub/platform/chipset/aspen/see/por.cmake` file.
    d. Add the algorithm directory to compile using the `add_subdirectory()` function in the `wm_proc/sensinghub/sensors/drivers/cmake_build/ssc_drivers.cmake` file.
5. After compiling the SWM code, verify that the algorithm registration function (specified by `register_func_name` in the algorithm SCons file) is added to `ssc_static_algorithms_list[]` array in the `wm_proc/zephyr/build/lpai_swm_img/arm/aspen.swm.prod/modules/sensinghub/sensors/algorithms/ssc_static_algorithms.c` file.

Application processor side changes

1. If the sensor driver is for a new sensor type that's different from the QTI reference design sensor types and if the sensor needs to be exposed to the sensor HAL, Android framework, and Android app then:
    a. Check if the sensor HAL source code has associated `cpp` file of the required new sensor type at <hlos_workspace>/vendor/qcom/proprietary/sensors-android/sensors-hal-2.X/sensors.
    b. If the `cpp` file for required new sensor type doesn't exist, then check with the vendor to receive the file. If the vendor doesn't have such file, then see the existing `gravity.cpp` or `step_count.cpp` file and create a new CPP file to support the new sensor type.
2. If the algorithm has custom `.proto` protobuf file:

   a. Copy the file to <hlos_workspace>/vendor/qcom/proprietary/sensors-ship/api/proto directory.
   b. Push it to the /vendor/etc/sensors/proto/ device directory.
3. If the algorithm has any JSON configuration file:
   a. Check if the configuration section at start of each driver configuration JSON file contains the expected values for hw_platform and platform_family (if present) items. The values must match the following command output. If the value from the command output isn't present in the respective item, then add the value to the respective item:
      ```
      adb shell cat /sys/devices/soc0/hw_platform
      adb shell cat /sys/devices/soc0/chip_family
      ```
   b. Add the JSON file name into /vendor/etc/sensors/hub2/config/json.lst.
   c. Push the modified configuration JSON files to the /vendor/etc/sensors/hub2/config/ device directory.
4. Reboot the device.
5. After rebooting, verify that the updated algorithm registry files are present in /mnt/vendor/persist/sensors/hub2/sensors/registry/registry/.

Verify sensor

See Tools to verify sensors.

2.1.4 Disable SWM QTI reference design sensor

This section is applicable only if the existing QTI reference design sensor driver or algorithm needs to be disabled.

- To disable the QTI reference design driver:
  – Locate the existing driver's CMakeLists.txt file and look for the name used as target under all occurrences of the add_ssc_su() function.
  – Remove the name(s) from include_sensor_vendor_libs list in the wm_proc/sensinghub/platform/chipset/aspen/see/por.cmake file. For example, to disable lsm6dsv driver remove it by modifying the wm_proc/sensinghub/platform/chipset/aspen/see/por.cmake file, as shown below.

```
+list(APPEND include_sensor_vendor_libs "sns_ak0991x" "sns_bmp5"
"sns_ct7117x" "sns_opt3001" "sns_max86178" "sns_da_test")
```

  – Remove the driver's directory from compilation in the wm_proc/sensinghub/sensors/drivers/cmake_build/ssc_drivers.cmake file. For example, to disable lsm6dsv driver modify the wm_proc/sensinghub/sensors/drivers/cmake_build/ssc_drivers.cmake file, as shown below.

```
-add_subdirectory(../lsm6dsv/build lsm6dsv)
```

  - Compile the SWM code and verify that the driver registration function (specified by `register_func_name` in the driver SCons file) isn't a part of `sns_static_drivers_list[]` array in the `wm_proc/zephyr/build/lpai_swm_img/arm/aspen.swm.prod/modules/sensinghub/sensors/drivers/sns_static_drivers.c` file.

- To disable the QTI reference design algorithm:
  - Locate the existing algorithm's `CMakeLists.txt` file and look for the name used as target under all occurrences of the `add_ssc_su()` function.
  - Add the name(s) as a new entry into the `exclude_libs` list in `wm_proc/sensinghub/platform/chipset/aspen/see/por.cmake` file. For example, to disable the QTI GameRV algorithm, following change must be applied.

```
+list(APPEND exclude_libs "sns_game_rv" "sns_game_rv_algo")
```

  - Remove the QTI algorithm's directory from compilation in the `wm_proc/sensinghub/sensors/drivers/cmake_build/ssc_algorithms.cmake` file. For example, to disable the QTI GameRV algorithm modify the `wm_proc/sensinghub/sensors/drivers/cmake_build/ssc_drivers.cmake` file, as shown below.

```
-add_subdirectory(../game_rv/build game_rv)
```

  - Compile the SWM code and verify that the algorithm registration function (specified by `register_func_name` in the algorithm SCons file) isn't a part of `sns_static_algorithms_list[]` array in the `wm_proc/zephyr/build/lpai_swm_ img/arm/aspen.swm.prod/modules/sensinghub/sensors/algorithms/sns_static_algorithms.c` file.

2.1.5 Convert SCons to CMake

SWM software supports CMake based compilation. Hence, if the third-party QSH driver or algorithm has SCons build file but not the `CMakeLists.txt` file, then it is required to convert the SCons file into the new `CMakeLists.txt` file.

1. As an example see the existing QTI reference design driver or algorithm `CMakeLists.txt` file to understand the usage of macros, `if()..else()...endif()` conditions, and others.
2. Create a new `CMakeLists.txt` file under the third-party driver or algorithm build directory.
3. Convert the third-party driver or algorithm SCons into the `CMakeLists.txt` file. While porting the changes, you can refer to the official documentation Writing CMakeLists Files, for the syntax of `CMakeLists.txt` file.
4. Replace the `env.AddSSCSU` function with `add_ssc_su`.

2.2 LPAIDSP QSH sensor integration

This section covers customization steps for LPAIDSP.

2.2.1 Customize QTI reference design sensor

This section is applicable only if the Electromyography/Electroneurography (EMG/ENG) sensor part used is the same as the SW6100/SW6100P QTI reference design but the sensor GPIOs, interrupt pin, or bus type used are different. See the *Registry in SEE* section of Sensors Execution Environment (SEE) Sensors Deep Dive while performing the following steps.

LPAIDSP side changes

If the GPIOs or bus interface type used for sensor is different from the QTI reference design sensor, then perform the following to configure the QUP instance as required.

- See SW6100/SW6100P Device Interfaces Guide to check if the `SSC QUP SE#` serial engine supports the intended bus interface type. If required, modify the intended QUP instance and/or serial engine configuration.

> **Note:**
>
> For the QUP instance and serial engine configuration support, raise a Salesforce case with Buses team with problem area as *BSP/HLOS, Driver/Peripheral, Peripheral Buses(UART/I2C/SPI/TSIF/etc) - non AP*.

Application processor side changes

Modify the sensor driver JSON configuration:

- Use the following table to locate the QTI reference design sensor's JSON config file in `/vendor/etc/sensors/hub1/config/` directory and pull the file from the device.

Sensor type	Part (Vendor)	JSON configuration file
Electromyography/Electroneurography (EMG/ENG)	STENG1AX (ST)	`aspen_steng1ax.json`

- Get the QUP instance ID from `bus_instance` registry item of the driver.
- Set `bus_type` to match the bus interface type.
- If the sensor uses a SPI or I^2C interface, then set `dri_irq_num` to the GPIO number used for the interrupt.
- Set `slave_config` to *Slave Address* for I^2C/I^3C bus interface or *Chip Select Number* for the SPI bus interface.
- Set `min_bus_speed_khz` to minimum bus speed supported by the sensor. Follow similar steps as described to set `max_bus_speed_khz`.
- Set `max_bus_speed_khz` to match maximum bus speed supported as per the sensor Data Sheet and the maximum speed supported by the bus interface of QUP serial engine.
  i. To know the QUP serial engine's maximum frequency for bus interface type, see SW6100/SW6100P Device Interfaces Guide.
  ii. To know the sensor hardware maximum bus interface frequency support, see the corresponding sensor Data Sheet.

     The following table lists the maximum supported frequency for sample QUP serial engines. (This is only for reference. See the above mentioned documents to confirm the values.)

Bus interface	Maximum frequency supported
I^3C	12.5 MHz
I^2C	10 MHz
SPI	50 MHz

Typically, the following sensor hardware's maximum bus interface frequency is supported:

Bus interface	Maximum frequency supported
I^3C	12.5 MHz
I^2C	400 KHz
SPI	10 MHz

Based on this example, following is the appropriate value for `max_bus_speed_khz`:

Bus interface	Sensor registry value
I^3C	12500
I^2C	400
SPI	9600

- Push the modified configuration JSON files to the `/vendor/etc/sensors/hub1/config/` device directory.

- Remove all the existing sensor driver registry files from the device. To perform this step, locate the file name prefix in the driver JSON configuration file as mentioned below and then remove all the registry files with the same prefix under `/mnt/vendor/persist/sensors/hub2/sensors/registry/registry/` directory.

```
"config": {
    ...
},
"prefix": {
```

- Reboot the device.
- After rebooting, verify that the updated driver registry files are present in `/mnt/vendor/persist/sensors/hub1/sensors/registry/registry/`.

Verify sensor

See Tools to verify sensors.

2.2.2 Integrate third-party sensor drivers

Perform the following steps to integrate third-party sensor drivers on the LPAIDSP. See the *Registry in SEE* section of Sensors Execution Environment (SEE) Sensors Deep Dive while performing the following steps.

LPAIDSP side changes

1. Acquire the directory from vendor, which contains the driver, associated SCons build file, JSON configuration files for driver, and any custom protobuf file for driver.
2. Place the acquired directory in `adsp_proc/ssc_drivers`.
3. If the driver has custom `.proto` protobuf file, then copy the file to `adsp_proc/qsh_api/pb`.
4. To enable the sensor driver for compilation, add driver SCons file name (without the SCons extension) into `include_sensor_vendor_libs.extend` array in the `adsp_proc/qsh_platform/chipset/aspen/qsh/por.py` file.
5. To enable the Island support for the driver, see the driver SCons file to find a flag with `SNS_ISLAND_INCLUDE_` keyword. Then add the rule to enable the same flag into `adsp_proc/qsh_platform/chipset/aspen/qsh/por.py` file.

   For example: `env.AddUsesFlags(['SNS_ISLAND_INCLUDE_XXXX'])`
6. If the bus interface GPIOs or bus interface type used for sensor is different from the QTI reference design sensor, then perform the following to configure the QUP instance as required.
   - See SW6100/SW6100P Device Interfaces Guide to check whether the `SSC QUP SE#` serial engine supports the desired bus interface type. If required, modify the intended QUP instance and serial engine configuration.

> **Note:**
>
> For any QUP instance and serial engine configuration support, raise a Salesforce case with the Buses team with problem area as *BSP/HLOS, Driver/Peripheral, Peripheral Buses(UART/I2C/SPI/TSIF/etc) - non AP*.

7. After compiling the LPAIDSP code, verify that the driver registration function (specified by `register_func_name` in the driver SCons file) is added to `sns_static_drivers_list[]` array in the `adsp_proc/qsh_platform/framework/build/sensor_img/qdsp6/aspen.adsp.prod/sns_static_drivers.c` file.

Application processor side changes

1. If the sensor driver is for a new sensor type that's different from the QTI reference design sensor types and if the sensor needs to be exposed to sensor HAL, Android framework, and Android app then:
   a. Check if the sensor HAL source code has associated CPP file of the required new sensor type at `<hlos_workspace>/vendor/qcom/proprietary/sensors-android/sensors-hal-2.X/sensors`.
   b. If the CPP file for required new sensor type doesn't exist, then check with the vendor to receive the file. If the vendor doesn't have such file, then see the existing `electro_neuro_graph.cpp` or `ambient_light.cpp` file and create a new CPP file to support new sensor type.
2. If the driver has custom `.proto` protobuf file, then:
   a. Copy the file to `<hlos_workspace>/vendor/qcom/proprietary/sensors-ship/api/proto`.
   b. Push it to `/vendor/etc/sensors/proto/` device directory.
3. Update the driver registry files received from the sensor vendor.
   a. Check if the configuration section at start of each driver configuration JSON file contains expected values for `hw_platform` and `platform_family` (if present). The values must match the following command output. If the value from command output isn't present in the respective item, then add the value to the respective item:
   ```
   adb shell cat /sys/devices/soc0/hw_platform
   adb shell cat /sys/devices/soc0/chip_family
   ```
   b. Use the QUP instance ID as `bus_instance` registry item value of the driver.
   c. Set `bus_type` item value to match the bus interface type.
   d. If the sensor uses a SPI or I^2C interface, then set `dri_irq_num` to the GPIO number used for the interrupt.
   e. Set `slave_config` item value with *Slave Address* for I^2C/I^3C bus interface or *Chip Select Number* for SPI bus interface.

f. Set the `min_bus_speed_khz` item with minimum bus speed supported by the sensor. Follow similar steps as described for setting the `max_bus_speed_khz` item.

g. Set `max_bus_speed_khz` to match supported maximum bus speed as per the corresponding *sensor Data Sheet* and maximum speed supported by the bus interface of QUP serial engine.

   i. To know the QUP serial engine's maximum frequency for bus interface type, see SW6100/SW6100P Device Interfaces Guide (80-74841-87).

   ii. To know the sensor hardware maximum bus interface frequency support, see the corresponding *sensor Data Sheet*.

   The following table lists the maximum supported frequency for sample QUP serial engines. (This is only for reference. See the above mentioned documents to confirm the values.)

Bus interface	Maximum frequency supported
I^3C	12.5 MHz
I^2C	10 MHz
SPI	50 MHz

Typically, the following sensor hardware's maximum bus interface frequency is supported:

Bus interface	Maximum frequency supported
I^3C	12.5 MHz
I^2C	400 KHz
SPI	10 MHz

Based on this example, following is the appropriate value for `max_bus_speed_khz` item:

Bus interface	Sensor registry value
I^3C	12500
I^2C	400
SPI	9600

4. Add the JSON file name into `/vendor/etc/sensors/hub1/config/json.lst`.
5. Push the modified configuration JSON files to the device `/vendor/etc/sensors/hub1/config/`.
6. Reboot the device.
7. After rebooting, verify that the updated driver registry files are present in `/mnt/vendor/persist/sensors/hub1/sensors/registry/registry/`.

Verify sensor

See Tools to verify sensors.

2.2.3 Integrate third-party provided algorithm

To integrate the third-party sensor algorithms on the LPAIDSP:

LPAIDSP side changes

1. Acquire the directory from vendor, which contains the sensor algorithms, associated SCons build file, any custom prototype file for algorithms, and the JSON configuration files, if any.
2. Place the acquired directory in `adsp_proc/qsh_algorithms`.
3. If the driver has custom `.proto` protobuf file, then copy the file to `adsp_proc/qsh_api/pb`.
4. To enable the sensor driver for compilation, add driver SCons file name (without the SCons extension) into `include_qsh_algorithms_libs.extend` array in the `adsp_proc/qsh_platform/chipset/aspen/qsh/por.py` file.
5. To enable the Island support for the driver, see the algorithm SCons file to find a flag with the `SNS_ISLAND_INCLUDE_` keyword. Then add the rule to enable the same flag into `adsp_proc/qsh_platform/chipset/aspen/qsh/por.py` file.
   For example: `env.AddUsesFlags(['SNS_ISLAND_INCLUDE_XXXX'])`
6. After compiling the LPAIDSP code, verify that the algorithm registration function (specified by `register_func_name` in the driver SCons file) is added to `qsh_static_algorithms_list[]` array in the `adsp_proc/qsh_platform/framework/build/sensor_img/qdsp6/aspen.adsp.prod/qsh_static_algorithms.c` file.

Application processor side changes

1. If the sensor driver is for a new sensor type that's different from the QTI reference design sensor types and if the sensor needs to be exposed to sensor HAL, Android framework, and Android app then:
   a. Check if the sensor HAL source code has associated CPP file of the required new sensor type at `<hlos_workspace>/vendor/qcom/proprietary/sensors-android/sensors-hal-2.X/sensors`.
   b. If the CPP file for required new sensor type doesn't exist, then check with the vendor to receive the file. If the vendor doesn't have such file, then see the existing `gravity.cpp` or `step_count.cpp` file and create a new CPP file to support the new sensor type.
2. If the algorithm has custom `.proto` protobuf file, then:
   a. Copy the file to `<hlos_workspace>/vendor/qcom/proprietary/sensors-ship/api/proto`.
   b. Push it to `/vendor/etc/sensors/proto/` device directory.
3. If the algorithm has any JSON configuration file:
   a. Check if the configuration section at start of each driver configuration JSON file contains expected values for `hw_platform` and `platform_family` (if present). The values

must match the following command output. If value from command output isn't present in the respective item, then add the value to the respective item:

```
adb shell cat /sys/devices/soc0/hw_platform
adb shell cat /sys/devices/soc0/chip_family
```

b. Add the JSON file name into `/vendor/etc/sensors/hub1/config/json.lst`.

c. Push the modified configuration JSON files to the device `/vendor/etc/sensors/hub1/config/` directory.

4. Reboot the device.
5. If the algorithm has any JSON config file, verify that the updated algorithm registry files are present in `/mnt/vendor/persist/sensors/hub1/sensors/registry/registry/`.

Verify sensor

See Tools to verify sensors.

2.2.4 Disable LPAIDSP QTI reference design sensor

This section is applicable only if the existing QTI reference design sensor driver needs to be disabled.

- Modify `adsp_proc/qsh_platform/chipset/aspen/qsh/por.py` file to remove the corresponding driver. For example, to disable `STENG1AX` driver delete `steng1ax` from the following statement:

```
include_sensor_vendor_libs.extend(['sns_da_test','steng1ax'])
```

- Compile the LPAIDSP code and verify that the driver registration function (specified by `register_func_name` in the driver SCons file) isn't a part of `sns_static_drivers_list[]` array in the `adsp_proc/qsh_ platform/framework/build/sensor_img/qdsp6/aspen.adsp.prod/sns_static_drivers.c` file.

2.3 RSB integration

As mentioned in the Rotary side button section, RSB sensor handling differs between Interactive and Ambient modes. In the Interactive mode, the application processor manages the sensor, whereas in the Ambient mode, the AWM takes over via the RSB offload. This involves a shared QUP interface over eGPIO, a common LDO for power control, separate drivers for each mode, and an entry/exit handshake between AP and AWM.

The following table lists the GPIO pins in the QTI reference design, which are allocated for the RSB sensor:

RSB pin name	GPIO#	SSC# / LPSS#
RSB_I2C_SDA	GPIO_72	SSC_24

RSB pin name	GPIO#	SSC# / LPSS#
RSB_I2C_SCL	GPIO_40	SSC_25
RSB_RESET_N	GPIO_99	LPASS_9
RSB_INT	GPIO_102	SSC_42

2.3.1 Configure RSB in Interactive mode

Enable TOP QUP SE8 in the APPS:

- Driver path – `vendor/qcom/opensource/coproc-kernel/rsb/`
- Key device tree files:
  - `vendor/qcom/opensource/coproc-devicetree/vienna/vienna-coproc.dtsi`
    - For interrupt and reset pins, `compatible = "pixart,pat9401"` specifies the device for driver probing.
  - `kernel_platform/msm-kernel/arch/arm64/boot/dts/vendor/qcom/vienna-pinctrl.dtsi`
    - `qupv3_se8_i2c_pins` – Used for I^2C pin control.
    - `qcom,apps` – Grants control to the APPS subsystem. This occurs during runtime when APPS resumes via `get_sync()`.
    - `qcom,remote` – Grants control to the SSC subsystem. This occurs during runtime when APPS suspends via `put_sync()`.

2.3.2 Configure RSB in Ambient mode

Enable SSC QUP SE10 in AWM.

- Driver path – `wm_proc/productsw/rsb_service/`.
- Key device tree file – `wm_proc/productsw/rsb_service/config/dts/pat9401e1.overlay`.

2.3.3 RSB offload policy configuration

The RSB functionality can be offloaded to AWM to operate in the Ambient mode. For each offload type: touch, RSB, and button, a policy is defined through a corresponding system property. These policies determine how the offload operations are handled:

Policy	Action	Notes
no_offload	No offload is performed.	–
optional	Offload is initiated, but no verification is performed on its success.	–
mandatory	Offload is initiated and verified for successful completion.	As verification is performed, this policy takes more time than the optional policy.

Use the `persist.vendor.rsb_offload` property, to check or set the current policy for RSB:

- Read policy – `adb shell getprop persist.vendor.rsb_offload`
- Set policy – `adb shell setprop persist.vendor.rsb_offload optional`

For more information about the display offload functionality, see SW6100/SW6100P Linux Android Display Technical Overview.

2.3.4 RSB test interfaces

You can use sysfs to enable or disable the RSB sensor and offload control for testing:

- Enable the RSB sensor (Default): echo 1 > /sys/class/rsb/enable.
- Disable the RSB sensor: echo 0 > /sys/class/rsb/enable.
- Enable the RSB offload to AWM: echo 1 > /sys/class/rsb/offload.
- Disable the RSB offload (Default): echo 0 > /sys/class/rsb/offload.

You can also read and write the PixArt PAT9401 sensor registers using sysfs for testing and validation:

- Read a register:
  - echo "r <register_address>" > /sys/class/input/input<index>/test
  - cat /sys/class/input/input<index>/test

  For example, to read the 0x0A register:

  - echo "r 0x0A" >/sys/class/input/input<index>/test
  - cat /sys/class/input/input<corresponding index>/test
  - Output – 0x0A: 0xFF
- Write a register:
  - echo "w <register_address> <value>" >/sys/class/input/input<index>/test

  For example, to write 0xFF to the 0x0A register:

  - echo "w 0x0A 0xFF" > /sys/class/input/input<index>/test

2.4 Sensor axis mapping and calibration

See How to Calibrate Sensors, for axis mapping and calibration steps of each sensor type.

2.5 Build and sideloading

This section provides references to software build steps and image sideloading procedure for sensor wear microcontroller (SWM) and LPAI digital signal processor (LPAIDSP).

2.5.1 Build SWM and LPAIDSP

See the release notes to know the host machine build environment and build commands for SWM and LPAIDSP software.

2.5.2 Sideload SWM and LPAIDSP

See *Sideloading the images* section of SW6100/SW6100P Software User Guide, for the procedure to sideload SWM and LPAIDSP images.

2.6 Memory usage

This section provides methods to identify memory usage at the module level in sensor wear microcontroller (SWM) and LPAI digital signal processor (LPAIDSP) images.

2.6.1 SWM image memory usage

The memory report generated in SWM is based on the facilities of Zephyr build system. The SWM memory report file is in `wm_proc/config/bsp/lpai_swm_img/build/aspen.swm.prod/LPAI_SWM_IMG_aspen.swm.prod_memory_report.txt` file.

2.6.2 LPAIDSP image memory usage

The LPAIDSP sensor PD memory report files are generated at `adsp_proc/config/bsp/sensor_img/build/aspen.adsp.prod`.

- `Memory_info_aspen_SensorsPD.csv` – Provides information about the non-Island modules.
- `Memory_info_aspen_SensorsPD_island.csv` – Provides information about the Island modules.

3 Develop sensors

This section explains how to develop sensor software across the application processor, sensor wear microcontroller (SWM), and LPAI digital signal processor (LPAIDSP). It describes how to use Qualcomm® sensing hub (QSH) APIs, understand source code organization, and implement sample algorithms and use cases.

3.1 Application processor

This section provides sensor-related details on the application processor side.

3.1.1 Source code structure

Sensors related code on the applications processor side is located at the following locations:

- `<hlos_workspace>/vendor/qcom/proprietary/sensors-ship` directory

```
vendor/qcom/proprietary/sensors-ship/
├─ api - QSH API component for all public proto APIs
├─ build_config - Build config directory
├─ config - Registry JSON config
├─ core - QSH ISession core
├─ lookup - QSH ISession SUID lookup
├─ services - sscrpcd Service
├─ test - QSH native test apps
└─ utils - QSH utils
```

- `<hlos_workspace>/vendor/qcom/proprietary/sensors-android` directory

```
vendor/qcom/proprietary/sensors-android/
├─ build_config - Build config directory
├─ sensors-cal-service - Sensor calibrate module
├─ sensors-hal-2.X - QSH sensors HAL
└─ test - Test application: USTA, QSTA
```

- `<hlos_workspace>/vendor/qcom/opensource/sensing-hub` directory

```
vendor/qcom/opensource/sensing-hub/
├─ apis - QSH API component for all public proto APIs
├─ build_config - Build config directory
├─ sensordaemon/init.vendor.sensors.rc - Sets persist registry directory
permissions
├─ common - Common header
├─ examples - QSH client API sample application
└─ session - QSH client (ISession) interface
```

3.1.2 QSH client APIs

The client must use the ISession APIs provided by QTI, to communicate with sensing hub to enable or disable sensors. The ISession APIs are available in the `<hlos_workspace>/vendor/qcom/opensource/sensing-hub/session/1.0/inc/` directory.

The QSH-exposed APIs are called QSH client APIs for applications. The client applications interact with the QSH framework available on the low-power processor. The following figure shows the detailed breakdown and usage of the QSH client APIs:

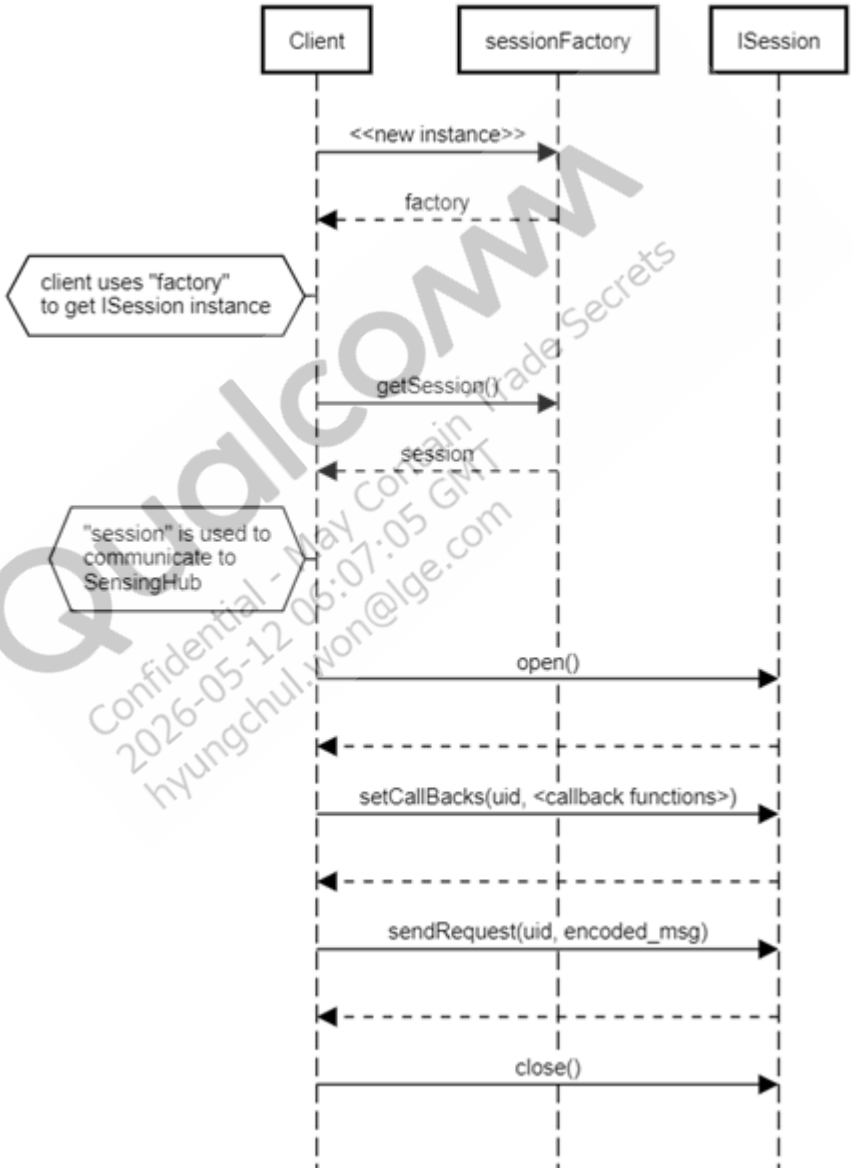


Figure: QSH client API

The following list describes the QSH client APIs:

getSession

To interact with the QSH, the sensor application client must create an interface session by invoking the `getSession()` API as shown in the figure. Successful creation of the interface session returns an `ISession*` object to the client.

Open, Close, SetCallback, and SendRequest

The interface created using the `getSession()` API operates on a *once_per_session* basis. This interface allows the application to use the QSH client APIs, such as `open()`, `close()`, `setCallback()`, and `sendRequest()`, until the session is explicitly deleted using the `close()` and `delete()` APIs.

After the interface session establishes, the application or the client code can open a sensor session by calling the `open()` API. Similar to `getSession()`, the `open()` API is also invoked on a *once_per_session* basis. Additionally, the `setCallback()` and `sendRequest()` APIs are invoked once for a specified SUID.

3.1.3 Sample client

The `SessionClient` sample application shows how to use QSH client API to develop applications. By default, this sample application builds in the `/usr/bin` directory on the device.

See `<hlos_workspace>/vendor/qcom/opensource/sensing-hub/examples/SessionClient/SessionClient.cpp` file, for the complete sample code.

3.2 Sensor wear microcontroller

This section provides sensor-related details on the sensor wear microcontroller (SWM) side.

3.2.1 Source code structure

`LPAI.WM.1.1-00xxx-LPAI_WM_PROD-y`, contains the SWM source code, where `xxx-y` represents the build number.

The following list shows the sensor-related directories and its usage in SWM source:

- `wm_proc/sensinghub` directory

```
wm_proc/sensinghub
├─ api - QSH API component for all public proto APIs
├─ config - sensing hub framework configuration
├─ drivers - QSH Context Aware Sensors - Location, BLE, Geofence
├─ platform - QSH Core framework, utilities, platform sensors
├─ sensors - QTI reference design sensor drivers
└─ zephyr - Build sensing hub as zephyr module
```

- `wm_proc/sensinghub/sensors` directory

```
wm_proc/sensinghub/sensors
├─ algorithms - QTI algorithm sensors
└─ drivers - QTI reference design physical drivers
```

- `wm_proc/sensinghub/driver` directory

```
wm_proc/sensinghub/sensors/drivers
├─ ak0991x - QTI reference design magnetometer sensor driver
├─ bmp5 - QTI reference design pressure sensor driver
├─ build - Build directory
├─ cmake_build - zephyr cmake build directory
├─ ct7117x - QTI reference design skin and ambient light sensor driver
└─ lsm6dsv - QTI reference design accelerometer/gyroscope sensor driver
```

3.2.2 QSH APIs

QSH APIs include the following:

- QSH sensor APIs – `wm_proc/sensinghub/api/pb and wm_proc/sensinghub/platform/api`
- QSH service APIs – `wm_proc/sensinghub/platform/inc/services`
- QSH context-aware sensor APIs – `wm_proc/sensinghub/drivers`

3.2.3 Sample algorithm

See the following directories for sample algorithms in the SWM QSH:

- For oem1 – `wm_proc/sensinghub/algorithms/oem1/` and `wm_proc/sensinghub/algorithms/qsh_oem1/` directories.
- For common source code of several QTI algorithms – `wm_proc/sensinghub/sensors/algorithms/` directory.
- For QNN proxy example – `wm_proc/sensinghub/algorithms/qnn_oem1/` directory.

See the QSH APIs mentioned in QSH APIs and the LPAIDSP sample algorithms mentioned in Sample algorithm.

To port the algorithm into SWM QSH:

1. Copy the LPAIDSP algorithm directory into SWM QSH.
2. See Convert SCons to CMake to convert SCons build script into CMake script.

3.2.4 Low-power Island mode

The SWM subsystem supports a low-power execution mode called low-power Island (LPI) mode that's more power efficient than the normal mode. LPI mode is different from the sleep mode. SWM can transition to sleep from either LPI mode or normal mode. On wake-up, SWM returns to the same mode that was active before sleep.

LPAIDSP in LPI mode

- Can't access DDR, but can access other internal memories within LPAI.
- Exhibits reduced power consumption.
- Supports system-level low power modes, such as XO shutdown and power-collapse, where DDR goes into self-refresh mode and other internal buses on SoC, which are required to access the non-Island modules are turned off.
- For better state management and reduced latency, either all or none of the subsystems in LPAI coverage enter the LPI mode.
- The application running in Island mode may vote for the Island mode exit in situations, such as transferring the data from Island memory backed buffer when it's near full to the DDR memory backed buffer, and sending data to the subsystem outside LPAI coverage.

LPAIDSP in Normal mode

- SWM can access DDR and other chip resources. This mode is used for interactions outside the LPAI covered subsystems.

Enable LPI support in SWM and QSH

- By default, the LPI mode is enabled in SWM.
- Entire QSH functionality internal to SWM doesn't depend on DDR.
- QSH doesn't classify the code specifically for LPI support in the SWM image.
- Files with the `_island` suffix in the QSH code are compiled the same way as other files.
- QSH uses the Glink transport mechanism to receive client requests or send data to clients.
- Glink uses Global TCM-backed buffers to communicate with LPAI subsystems and DDR-backed buffers to communicate with subsystems outside LPAI coverage.
- Glink client votes for the LPI mode exit when initiating glink TX on channels that use DDR-based shared memory.

3.2.5 Develop algorithm

See the QSH APIs mentioned in QSH APIs and the sample algorithms mentioned in Sample algorithm to develop a new algorithm. Additionally, see Convert SCons to CMake to convert SCons build script into CMake script.

3.3 LPAI digital signal processor

This section provides sensor-related details on the LPAI digital signal processor (LPAIDSP) side.

3.3.1 Source code structure

`LPAIDSP.HT.1.4-00xxx-ASPEN-y`, contains the LPAIDSP source code, where `xxx-y` represents the build number.

The following list shows the sensor-related directories and its usage in the LPAIDSP source.

```
adsp_proc/
├─ qsh_algorithms - QSH - All fusion algorithms
├─ qsh_api - QSH API component for all public proto APIs
├─ qsh_drivers - QSH Context Aware Sensors - WWAN, WiFi, Audio
├─ qsh_platform - QSH Core framework, utilities, platform sensors
└─ ssc_drivers - QTI reference design physical sensor drivers
```

3.3.2 QSH APIs

QSH APIs include the following:

- QSH sensor APIs – `adsp_proc/qsh_api/pb and adsp_proc/qsh_platform/api`
- QSH service APIs – `adsp_proc/qsh_platform/inc/services`
- QSH context-aware sensor APIs – `adsp_proc/qsh_drivers`

3.3.3 Sample algorithm

Reference algorithms, `oem1`, `qsh_oem1`, and `ai_oem1` are available to explore how the QSH APIs are used, and algorithms can request data from physical sensor or another algorithm. The reference algorithms are located at `adsp_proc/qsh_algorithms`.

- `oem1` algorithm: Demonstrates adding dependency of a physical sensor, sending request to physical sensors, processing received physical sensor data, sending request to registry sensor to read the registry items, and processing registry event.
- `qsh_oem1` algorithm: Demonstrates adding dependency of other algorithms, sending request to algorithms, and processing received algorithm data.
- `ai_oem1` algorithm: Demonstrates sensor data processing on eNPU using QNN framework.

To enable the sample algorithm, add the SCons file name of the algorithm (without SCons extension) to `include_qsh_algorithms_libs.extend` in the `adsp_proc/qsh_platform/chipset/aspen/por.py` file.

See Tools, to verify the sample algorithm.

3.3.4 Low-power Island mode

The LPAIDSP subsystem supports a low-power execution mode called low-power Island (LPI) mode that's more power efficient than the normal mode. LPI mode is different from the sleep mode. LPAIDSP can transition to sleep from either LPI mode or normal mode. On wake-up, LPAIDSP returns to the same mode that was active before sleep.

LPAIDSP in LPI mode

- LPAIDSP is self-sufficient to execute only from L2/TCM memory, with reduced Island-only functionality.
- During Island mode of operation, code and data needed for operation are copied from DDR to L2/TCM.
- Can't access DDR, but can access other internal memories within LPAI.
- Exhibits reduced power consumption.
- LPAIDSP runs in a power optimized way and hence at reduced speeds.
- Supports system-level low power modes, such as XO shutdown and power-collapse, where DDR goes into self-refresh mode and other internal buses on SoC, which are required to access the non-Island modules are turned off.
- For better state management and reduced latency, either all or none of the subsystems in LPAI coverage enter the LPI mode.
- The application running in Island mode may vote for the Island mode exit in situations, such as transferring the data from Island memory backed buffer when it's near full to the DDR memory backed buffer, and sending data to the subsystem outside LPAI coverage.

LPAIDSP in Normal mode

- LPAIDSP executes out of DDR memory, with all the functionalities.
- In Normal mode, LPAIDSP can access all external memories, like DDR and other chip resources. This mode is used for interactions outside the LPAI covered subsystems. Here, performance-intensive operations are possible; LPAIDSP can run at full speed and vote for on-chip resources.

Enable LPI support in SWM and QSH

- By default, the LPI mode is enabled in LPAIDSP and supported by QSH.
- Island code that's classified for Island support at compile time, is a subset of the entire image.
- QSH has classification of the code for LPI support in the LPAIDSP image.

- Files with the `_island` suffix in the QSH code are compiled for LPI mode support whereas the other files are compiled only for normal mode execution.
- To communicate with SWM:
  - QSH uses the Glink transport mechanism to receive client requests or send data to clients.
  - Glink uses Global TCM-backed buffers to communicate between subsystems.
  - LPAIDSP need not exit the LPI mode.
- To communicate with application processor:
  - QSH uses the QMI transport mechanism to receive client requests or send data to clients.
  - QMI uses DDR-backed buffers to communicate between subsystems.
  - QSH votes for the LPI mode exit.

3.3.5 Develop algorithm

See the QSH APIs mentioned in QSH APIs and the sample algorithms mentioned in Sample algorithm to develop a new algorithm.

3.4 Wear health service

QTI provides sample code and a test application for the wear health service (WHS) hardware abstraction layer (HAL)-level validation.

3.4.1 Source code structure

The WHS source code is stored in the following directories:

- WHS HAL – `<hlos_workspace>/vendor/qcom/proprietary/vienna/whs-sensors/`.
- SWM health offload sensor – `wm_proc/sensinghub/platform/sensors/health_offload`.

3.4.2 WHS APIs

Contact Google for WHS API details.

3.4.3 Sample reference

QTI provides a reference implementation in the SWM health offload sensor to integrate the QTI pedometer algorithm for the WHS use case. The reference implementation is available in `wm_proc/sensinghub/platform/sensors/health_offload/src/sns_health_offload_reference.c`.

3.4.4 Test application

WHS Android app

For testing it's recommended to use Google provided WHS Android app or OEM/third-party developed WHS Android app.

WHS HAL native test application

QTI provides a native test application only for the test and debug purpose to confirm the WHS HAL functionality. It can be used in a situation to narrow down a problem and identify whether the issue is present in WHS HAL or the above layers, such as WHS Android app or WHS service.

- WHS HAL test application – `vendor/qcom/proprietary/vienna/whs-sensors/whs-hal-2.0/test/`
- Test command

```
whs_hal_test <optional_delay_value_in_seconds>
```

- Usage – The application executes hard-coded WHS APIs. By default, it runs various APIs with a 10 seconds interval. Provide a new delay value (in seconds) using the first parameter to override the default. OEM needs to customize the application for any other use-case.

3.4.5 Develop

OEMs must complete the following integration tasks to support WHS on their devices.

- Integrate all algorithms required by the WHS specification. WHS algorithms can be integrated into the QSH sensor framework like any other QSH algorithm. See Integrate third-party provided algorithm or Develop algorithm, for information to integrate a custom algorithm.
- Support features in the custom algorithm for auto-start, auto-pause, and auto-stop functionality.
- Adjust the WHS metric data format for correct decoding if required:
  - Use the predefined hard-coded mapping of metric data type to the intended event format in the `datatype_output_format()` function in `vendor/qcom/proprietary/vienna/whs-sensors/whs-hal-2.0/framework/inc/whs_types.h`.
  - Each metric type is associated with a specific output format type.

The following table describes OEM responsibilities for each WHS API.

OEM responsibilities with respect to the API functionality

WHS API	OEM responsibility
`getSupportedMetricsMapping()`	Update OEM configuration XML file for required data/metric types.
`getSupportedGoals()`	Update OEM configuration XML file with threshold values.
`getAutoStartCapabilities()` `getAutoStopEnabledExerciseTypes()` `getAutoPauseEnabledExerciseTypes()`	Update OEM configuration XML file with exercises that support auto-start, auto-stop, and auto-pause functionality.
`setDataListener()`	None.
`IHealthServicesCallback.onDataUpdate()`	Consume and use the data as expected.
`IHealthServicesCallback.onFlushCompleted()`	Consume and use the data as expected.
`setProfile()`	Extend health offload sensor to broadcast data to all or selected algorithms. The algorithm must be compliant for consuming this data.
`addGoal() removeGoalById()`	Extend health offload sensor for required/additional algorithms.
`IHealthServicesCallback.onGoalAchieved()`	Consume and use the data as expected.
`updateTrackingConfig()`	None.
`IHealthServicesCallback.onAvailabilityUpdate()`	Publish the associated algorithm as unavailable/available so the health offload sensor knows when it recovers after a failure.
All other APIs	Extend health offload sensor to use the API and algorithm must be compliant to understand the data.

3.5 Sports sensor offload

3.5.1 Source code structure

The SSO source code is stored in the following directories:

- SSO HAL – It's shipped in library form.
- SWM formatter sensor – `wm_proc/sensinghub/platform/sensors/formatter`.

3.5.2 Sidekickmetric APIs and SDK

The `Sidekickmetric` APIs and SDK are stored in the following directories:

- `Sidekickmetric` APIs – `<qxsi_hlos_workspace>/vendor/qcom/proprietary/vienna/wear-interfaces/sidekickmetrics`.
- Offload SDK – For SSO application development, QTI provides the Offload SDK at `<qxsi_hlos_workspace>/vendor/qcom/proprietary/commonsys/android-weartech-core/UxOffload/sdk`.

3.5.3 Sample reference

The SSO sample application uses `Sidekickmetric` APIs for sensor data offload and `UxOffload` APIs for display offload.

Code locations:

- Sample application – `<qxsi_hlos_workspace>/vendor/qcom/proprietary/commonsys/android-weartech-app/WearTechWatchUI`
- SWM formatter sensor – QTI provides a reference implementation in the SWM formatter sensor to integrate the QTI pedometer algorithm for the SSO use case. The reference implementation is available in `wm_proc/sensinghub/platform/sensors/formatter/src/sns_health_offload_reference.c`.

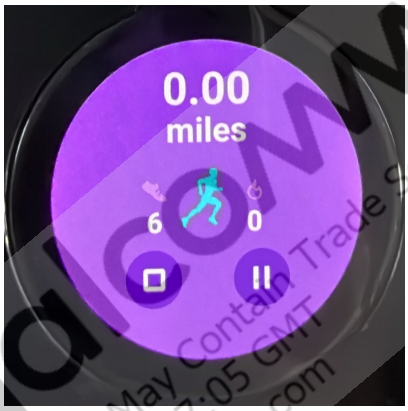


Figure: Interactive mode

Figure: Ambient mode

3.5.4 Develop

To add a new supported metric algorithm, perform the following steps:

1. To develop the SSO Android app, see Sidekickmetric APIs and SDK and Sample reference.
2. To add a new supported metric algorithm, add the required metric algorithm sensor type to `device/qcom/common/ssohal/metric_sensors_list.txt`.
   a. See Integrate third-party provided algorithm or Develop algorithm, for information to integrate a custom metric algorithm.
   b. In `<qxsi_hlos_workspace>/vendor/qcom/proprietary/commonsys/android-weartech-core/UxOffload/OffloadSensorService/src/com/qualcomm/qti/offloadsensorservice/SensorServiceConstants.java`, modify `convertSensorInfo()`, `typeToDataSource()`, and `typeToformat()` functions to support the new metric algorithm.
   c. In the `metric_handler_table` global array in `wm_proc/sensinghub/platform/sensors/formatter/src/sns_formatter_reference.c` formatter file, perform the following:
      - Add the new metric algorithm sensor type to the array.
      - Check the reference implementation and implement new functions to handle the new metric algorithm sensor type.

Note:

The QSH Display bridge doesn't require any code changes to handle new metric algorithm sensor types.

3.6 Wrist/Watch orientation

3.6.1 Source code structure

The following table describes the software modules involved in WO for each product.

Software product	Modules	
	APPS	SWM
LW	WO service and WO HAL at: `<hlos_workspace>/vendor/qcom/proprietary/libWristOrientation`	WO service at: `wm_proc/productsw/lpi_services/wm_utils`
LAW	WO service and WO HAL at: `<hlos_workspace>/vendor/qcom/proprietary/libWatchOrientation`	

3.6.2 QSH APIs

See wm_proc/sensinghub/api/pb/sns_device_mode.proto file for details about device_mode sensor APIs.

3.6.3 Sample reference

For reference the QTI Wrist Tilt algorithm source is at wm_proc/sensinghub/sensors/algorithms/wrist_tilt, with orientation mode change handling under the WRIST_TILT_ENABLE_DEVICE_MODE_SENSOR macro.

3.6.4 Develop

For an orientation-sensitive QSH sensor, follow these guidelines to handle WO mode change:

- From the QSH sensor instance, create a stream with the device_mode sensor type.
- As device_mode is an on-change sensor, send SNS_STD_SENSOR_MSGID_SNS_STD_ON_CHANGE_CONFIG request over the stream.
- After the client request is processed or when the orientation change occurs, the QSH instance receives SNS_DEVICE_MODE_MSGID_SNS_DEVICE_MODE_EVENT from the device_mode sensor.
- Decode the received event and continue processing the event only if the event mode is SNS_DEVICE_MODE_WEAR_WRIST_RIGHT or SNS_DEVICE_MODE_WEAR_DISPLAY_ROTATED, and the event state isn't SNS_DEVICE_STATE_UNKNOWN.
- Determine the orientation mode and make the necessary updates in the QSH sensor instance state to work with new orientation mode.

4 Test and troubleshoot sensors

This section describes tools and methods to validate sensor functionality and diagnose issues. It describes how to use Qualcomm® sensing hub (QSH) test tools to verify sensor availability, data flow, and basic operation.

4.1 Tools

The test tools ensure the accuracy, reliability, and optimal performance of both the hardware-based and software-based sensors.

4.1.1 Android applications

Unified sensor test application (USTA)

- USTA bypasses sensors HAL and directly communicates with the QSH framework on sensor wear microcontroller (SWM) and LPAI digital signal processor (LPAIDSP).
- USTA can be useful in early sensor bring-up phase where particular sensor isn't exposed in sensor HAL.
- USTA can send requests to respective sensors and decode received events.
- USTA supports the custom proto file.
- On wearables platform, USTA is supported only in the command line mode.
- See Unified Sensor Test Application (USTA) User Guide, for more information about USTA.

QSensor test application (QSTA)

- QSTA uses standard Android framework APIs to enable or disable sensors. Hence, all requests go through the sensors HAL.
- On wearables platform, QSTA is supported only in the command line mode.
- QSTA command line parameters:

Parameter	Purpose
`sensorname`	Name of the sensor
`sensortype`	Type of the sensor
`rate`	Sample rate for streaming sensor (in milliseconds)
`batch`	Batch period (in milliseconds)

- The following commands are used to enable and disable sensors using the command line.
  1. Start QSTA service

```
adb shell am start-foreground-service com.qualcomm.qti.
sensors.qsensortest/com.qualcomm.qti.sensors.core.
qsensortest.QSensorService
```

  2. Get list of all sensors

```
adb shell sqlite3 /data/data/com.qualcomm.qti.sensors.
qsensortest/databases/qsensortest.db 'select sensor_name
from Sensor'
```

Output format – `<sensor_name>.<sensor_type>`
Sample output:

```
lsm6dsv Accelerometer Non-wakeup.1
pedometer_wrist Non-wakeup.19
```

  3. Prepare command parameters
     A list of all the sensors is displayed in the previous step. Following format shows how to identify sensor name and sensor type in the output.
     According to the sample output in the previous step, the sensor name is `lsm6dsv Accelerometer Non-wakeup` and the sensor type is `1`.
     To enable and disable the sensor:
     – Replace all the white spaces in sensor name and use it as a value for the `sensorname` parameter.
     – Use the sensor type as a value for the `sensortype` parameter.
     – Streaming sensor:
       ○ To enable – Use a non-zero sample rate and the required batch period.
       ○ To disable – Use `-1` as the sample rate.
     – On-change sensor:
       ○ To enable – Use zero sample rate and the required batch period.
       ○ To disable – Use `-1` as the sample rate.
  4. Enable sensor
     – For streaming sensor

```
adb shell am broadcast -a com.qualcomm.qti.
sensors.qsensortest.intent.STREAM -e 'sensorname'
'lsm6dsv_Accelerometer_Non-wakeup' --ei
sensortype 1 --ei rate 60 --ei batch 60
```

     – For on-change sensor

```
adb shell am broadcast -a com.qualcomm.qti.
sensors.qsensortest.intent.STREAM -e 'sensorname'
'pedometer_wrist__Wakeup' --ei sensortype 19 --ei
rate 0 --ei batch 0
```

5. Disable sensor
   - For streaming sensor

     ```
     adb shell am broadcast -a com.qualcomm.qti.
     sensors.qsensortest.intent.STREAM -e 'sensorname'
     'lsm6dsv_Accelerometer_Non-wakeup' --ei
     sensortype 1 --ei rate -1
     ```

   - For on-change sensor

     ```
     adb shell am broadcast -a com.qualcomm.qti.
     sensors.qsensortest.intent.STREAM -e 'sensorname'
     'pedometer_wrist__Wakeup' --ei sensortype 19 --ei
     rate -1
     ```

6. Stop QSTA service

   ```
   adb shell am force-stop
   com.qualcomm.qti.sensors.qsensortest
   ```

4.1.2 Android native QSH test tools

The following tools facilitate comprehensive testing of the sensor functionalities.

- The sensor info test provides detailed information about the sensor.
- The driver acceptance test ensures that the sensor drivers are functioning correctly and are compatible with Qualcomm sensing hub (QSH).
- The sensor workhorse is a tool to stress-test the sensors under various conditions.

Use sensor info test to list supported sensors

The sensor info test `ssc_sensor_info` is an application within the QSH test suite. It lists the QSH supported sensors and their attributes. You can run the following query at high-level operating system (HLOS) to get the attributes of a specified data type.

```
ssc_sensor_info [-sensor=<sensor_type>] [-delay=<time_in_seconds>][-duration=
<time_in_seconds>][-default_only=<1|0>][-log=<1|0>][-help]
```

Table : Parameters of ssc_sensor_info

Flags	Type	Value	Units	Notes
Sensor	`string`	Sensor type.	–	Queries attribute information for the specified sensor type.
Log	`int`	0 | 1	–	Enables or disables diagnostic (API) logs.
help(h)	`int`	–	–	Displays command usage help.

Flags	Type	Value	Units	Notes
Duration	int	Positive values	seconds	Specifies the wait time for the sensor attribute updates.
Delay	int	Positive values	seconds	Specifies the time delay before sending the sensor requests.
default_only	int	0 | 1	–	If the default_only flag is set to False, UIDs of all the available sensors that support the specified data type are sent. If the default_only flag is set to True, only the UID of the default sensor availability is sent.

Sensor info test examples

- Query the sensor attributes and generate diagnostic logs.

```
ssc_sensor_info
```

- Query all *accel* sensor attributes.

```
ssc_sensor_info –sensor=accel
```

Use the driver acceptance test to validate sensor drivers

The ssc_drva_test driver acceptance test tool does the following:

- Performs sensor driver validation and operates at the QSH sensor API layer.
- Executes a range of sensor use cases, such as streaming the sensor data and batching a sensor at a selected sampling frequency.
- Accepts the parameters directly from the command line and eliminates the need for compile-time options.

This approach makes it a convenient and efficient tool to perform basic driver-level tests or validations at HLOS.

```
ssc_drva_test [-sensor=<sensor_type>] [-duration=<time_in_seconds>] [-sample_
rate=<value_in_Hz>] [-batch_period=<value_in_seconds>] [-iterations=<value>] [-
num_samples=<value>] [-factory_test=<value>]
```

The following table describes the parameters used by the ssc_drva_test test application.

Table : Parameters of ssc_drva_test

Flag	Type	Value	Unit	Notes
`sensor`	`string`	Sensor type.	–	Mandatory argument: limited to the available sensor types. See Use sensor info test to list supported sensors, to know the available sensor types.
`duration`	`float`	Positive values only	Sec	Mandatory argument: sensor test duration in seconds.
`sample_rate`	`float`	Positive floating point number values: • -1: Maximum sampling rate • -2: Minimum sampling rate	Hz	Mandatory for streaming sensors, optional for on-change sensors.
`batch_period`	`float`	Positive floating point numbers	Sec	This period is the same as the report period and indicates how long to buffer the samples and report outside of the low-power processor.
`iterations`	`int`	Positive values only	N.A.	Provides the number of times the test must be repeated.
`num_samples`	`int`	Positive values only	N.A.	Indicates the minimum number of samples intended to be collected. If a `num_samples` parameter is specified and the test does not collect enough samples during the test, the test sensor generates FAIL. The `num_samples` parameter forces the test to run for a maximum duration between a specified duration or duration calculated by the following: • `num_samples*` expected sample rate, where the expected sample rate is the rate at which the sensor is expected to serve.

Flag	Type	Value	Unit	Notes
factory_test	int	• 0 (SNS_PHYSICAL_SENSOR_TEST_TYPE_SW) • 1 (SNS_PHYSICAL_SENSOR_TEST_TYPE_HW) • 2 (SNS_PHYSICAL_SENSOR_TEST_TYPE_FACTORY) • 3 (SNS_PHYSICAL_SENSOR_TEST_TYPE_COM)	N.A.	Selects the type of factory test (from the value column) that you want to run.

Driver acceptance test examples

- Stream a single sensor at a selected sampling frequency for a known duration:

```
ssc_drva_test -sensor=accel -duration=5 -sample_rate=50
```

- Batch a single sensor at a selected sampling frequency and report period for a known duration:

```
ssc_drva_test -sensor=accel -duration=30.0 -sample_rate=50 -batch_period=2.
0
```

- Self-test for accelerometer (hardware self-test), which checks the health and basic functionality of the accelerometer sensor by running the built-in diagnostic routines:

```
ssc_drva_test -sensor=accel -factory_test=1 -duration=10
```

Output

On the console command line, this test outputs `PASS` or `FAIL`, which indicates only the test execution status.

Use the sensor workhorse test to perform stress test of sensors

The `see_workhorse` sensor workhorse tool does the following:

- Operates specific sensors based on the command-line arguments.
- Streamlines sensor testing and data collection in various configurations.

```
see_workhorse [-sensor=<sensor_type>][-sample_rate=<max | min | positive
integer value>] [-batch_period=<seconds>] [-calibrated=<0 | 1>] [-wakeup=<0 |
1>]
```

Table : Parameters of see_workhorse

Flags	Type	Value range	Units	Notes
`sensor`	`string`	Sensor type.	N.A.	Mandatory argument: Limited to the available sensor types. See Use sensor info test to list supported sensors, to know the available sensor types.
`on_change`	`int`	0 | 1	N.A.	• 0 selects using `SNS_STD_SENSOR_STREAM_TYPE_STREAMING`. • 1 selects using `SNS_STD_SENSOR_STREAM_TYPE_ON_CHANGE`.
`sample_rate`	`float`	Positive floating point number values: • -1: Maximum sampling rate • -2: Minimum sampling rate	Hz	Mandatory for streaming sensors, optional for on-change sensors.
`batch_period`	`float`	Positive floating point numbers	Seconds	Same as the batch period or report period.
`calibrated`	`int`	0 | 1	N.A.	• 0: nop (default). • 1: if the `sensor_type` is gyro or mag, then also activate `gyro_cal` or `/mag_cal`, respectively.

Flags	Type	Value range	Units	Notes
wakeup	int	0 | 1	NA	• 0: sets suspend_config wake-up to SEE_CLIENT_DELIVERY_NO_WAKEUP. • 1: sets suspend_config wake-up to SEE_CLIENT_DELIVERY_WAKEUP (default).
display_events	int	0 | 1	N.A.	Display sensor events in JSON format, using the event callbacks.

Sensor workhorse examples

For instance, use the following command to stream the accelerometer data at 50 Hz sample rate for 10 sec:

```
see_workhorse -sensor=accel -sample_rate=50 -duration=10 -display_events=1
```

4.2 Debug methods

This section describes the debug methods available for sensor issues. The following table depicts debug method per activity.

Debug activity	Debug method
Sensor HAL logs	Logcat
SWM QSH logs	• QXDM Professional • On-device Diag logging • Zephyr shell
LPAIDSP QSH logs	• QXDM Professional • On-device Diag logging
Crashes and complex issues on LPAIDSP and SWM	RAM dump

4.2.1 Sensor HAL logs

The sensor HAL default logcat logging is set to `INFO` level. To enable the `VERBOSE` level logging, run the following commands and reboot the device.

```
adb root
adb shell setprop persist.vendor.sensors.debug.hal v
adb reboot
```

4.2.2 WHS HAL logs

The WHS HAL default logcat logging is set to `INFO` level. To enable the `VERBOSE` level logging, run the following commands and reboot the device.

```
adb root
adb shell setprop persist.vendor.whs.hal v
adb reboot
```

4.2.3 SSO HAL logs

The SSO HAL default logcat logging is set to `INFO` level. To enable the `VERBOSE` level logging, run the following commands and reboot the device.

```
adb root
adb shell setprop persist.vendor.debug.metrics.hal v
adb reboot
```

4.2.4 QXDM Professional

- QXDM Professional supports live Diag logging using USB connection. Sensor modules can use the QSH provided macros and APIs to send debug print and log packets to QXDM Professional.
- See the *QXDM logging* section of SW6100/SW6100P Software User Guide, for the procedure for capturing the QXDM Professional log. While following the steps, use the following log masks for sensor-related logs.
  - Message Packets → SNS → All logs
  - Message Packets → IOT → WEAR_AON → SNS → All logs
  - Log Packets → Common → SNS → All logs

4.2.5 On-device diag logging

- On-device logging allows diagnostic traffic to be saved as a file on the device. It can be used in field testing when connecting the device to a workstation or laptop is undesirable. Later, host PC tools can be used to process the saved log file.
- See the *On-device logging* section of SW6100/SW6100P Software User Guide, for the procedure for capturing on-device diag logging. While performing the steps, use the following log masks for sensor-related logs.
  - Message Packets → SNS → All logs
  - Message Packets → IOT → WEAR_AON → SNS → All logs
  - Log Packets → Common → SNS → All logs

4.2.6 Logging over Zephyr shell (UART)

See the *Zephyr shell logging* section of SW6100/SW6100P Software User Guide, for the procedure to add log into Zephyr shell.

4.2.7 RAM dump

This method allows capturing dump of various memories, such as DDR, LPAIDSP TCM, and SWM TCM. The RAM dump is then parsed to extract the debug information, such as call stack, and log buffer.

See the *RAM dump mode and RAM dump collection* section of SW6100/SW6100P Software User Guide, for the procedure for capturing RAM dump.

5 Sensors customer support

This section outlines how to get customer and engineering support for sensor-related issues. It explains support channels, problem area selection, and guidance for working with sensor vendors.

QTI provides customer engineering support through the Salesforce portal, Qualcomm support.

For QSH sensor-related support, select the problem areas to direct the case to the appropriate Sensors team.

- Problem Area 1
  - Board Support Package (BSP)
- Problem Area 2
  - Sensors
- Problem Area 3
  - For AP side issues: Sensors – AP/HLOS
  - For non-AP side issues: Sensors Core – SSC

- For QSH context-aware sensors, select the following problem areas to direct the case to the appropriate technology team.

Technology	Audio	Location	BLE	Wi-Fi	WWAN
Problem Area 1	Multimedia	Wireless Connectivity Software			Modem Software
Problem Area 2	Audio (Multimedia)	gpsOne	Bluetooth	WLAN	QMI / RIL / Telephony / Android and Windows data
Problem Area 3	Choose according to the observed issue				Android RIL

5.1 Sensors vendor ecosystem

This section provides information about the sensor vendor ecosystem for sensor parts and algorithms other than those provided by QTI on the SW6100/SW6100P QTI platform.

- QTI provides complete hardware and software ecosystem for sensor vendors to develop QSH drivers and algorithms.

- For sensor driver
  - Hardware/Software platform: HDK8650 or later with OpenSSC 12.x package or later.
  - QTI recommends OEM to request vendor for the preferred vendor list (PVL) validation.
  - Validation and delivery: The sensor vendors can provide drivers to customers directly after complete validation. OEM can request the following from vendors for a driver delivery:
    - QTI-defined Driver Acceptance checklist listing the passing and failing test cases.
    - Compatibility test suite (CTS) test results.

Note:

QTI isn't committed to provide any support for issues that relate to, or a result of, use of sensor vendor drivers other than provided in the mainline software, and OEM should contact vendor for those issues. OEMs need to ask sensor vendors to provide fully validated drivers on the recommended HDK/OpenSSC platform and have sensor part fully tested and certified.

- For sensor algorithm - Contact QTI.

Note:

QTI isn't committed to provide any support for issues that relate to, or a result of, use of sensor vendor algorithm, on SW6100/SW6100P QTI platform, and OEM should contact vendor for those issues. OEMs need to ask sensor vendors to provide validated algorithm on the recommended HDK/OpenSSC platform or QTI reference device.

6 References

6.1 Related documents

Title	Number
Qualcomm Technologies, Inc.	
Sensors Execution Environment (SEE) Sensors Deep Dive	80-P9301-35
Sensors Execution Environment Client API	80-P9301-36
Unified Sensor Test Application (USTA) User Guide	80-P9301-85
Qualcomm® Sensing Hub (QSH) 4.0 API Reference	80-40939-19
SW6100/SW6100P Software User Guide	80-74841-50
SW6100/SW6100P Device Interfaces Guide	80-74841-87
SW6100/SW6100P Linux Android Display Technical Overview	80-74841-97

6.2 Acronyms and terms

Sensors-related acronyms and expansions for better understanding.

Acronym or term	Definition
aDSP	Application digital signal processor
AIDL	Android interface definition language
ALS	Ambient light sensor
AMD	Absolute motion detection
ARW	Activity recognition on wearables
AWM	Ambient wear microcontroller
BLE	Bluetooth low energy
CTS	Compatibility test suite
DRI	Data ready interrupt
DSP	Digital signal processor
ECG	Electrocardiogram
EMG	Electromyography
ENG	Electroneurography
eGPIO	Enhanced general-purpose input/output
eNPU	Embedded neural processing unit
GRV	Game rotation vector
HAL	Hardware abstraction layer
HLOS	High-level operating system
I^2C	Inter-integrated circuit
I^3C	Improved I^2C

Acronym or term	Definition
IBI	In-band interrupt
IMU	Inertial measurement unit
LAW	Linux Android wearables
LPAI	Low-power AI
LPAIDSP	LPAI digital signal processor
LPI	Low-power Island
LW	Linux wearables
ODR	Output data rate
PD	Protection domain
PPG	Photoplethysmogram
PSMD	Persistent stationary/motion detection
PVL	Preferred vendor list
QNN	Qualcomm neural network
QSH	Qualcomm sensing hub
QUP	Qualcomm universal peripherals
QuRT	Qualcomm real time operating system
RSB	Rotary side button
RTOS	Real time operating system
RV	Rotation vector
SCons	Software construction
SEE	Sensors execution environment
SMD	Significant motion detection
SoC	System-on-chip
SPI	Serial peripheral interface
SSC	Qualcomm® Snapdragon™ sensors core
SSO	Sports sensor offload
SSR	Subsystem restart
SUID	Sensor unique identifier
SWM	Sensor wear microcontroller
TCM	Tightly coupled memory
UART	Universal asynchronous receiver/transmitter
WDP	Wearables development platform
WHS	Wear health service
WM	Wearables Microcontroller
WO	Wrist/Watch orientation
WRD	Wearables reference design

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this "Material"), is subject to your (including the corporation or other legal entity you represent, collectively "You" or "Your") acceptance of the terms and conditions ("Terms of Use") set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. ("**Qualcomm Technologies**"), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as "**Qualcomm Internal Use Only**", no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies' prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the Website Terms of Use on www.qualcomm.com, the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, or other legal statements or notices found on prior pages of the Material, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.

THE DOCUMENTATION ACCOMPANYING THE MATERIALS AND/OR RELEVANT PRODUCTS AND SERVICE OFFERINGS MAY INCLUDE IMPORTANT USE LIMITATIONS. ANY DEVIATIONS FROM APPLICABLE USE LIMITATIONS MAY ADVERSELY IMPACT PERFORMANCE, DURABILITY, QUALITY OR SAFETY. YOU ASSUME ALL RISKS AND LIABILITIES ASSOCIATED WITH ANY DEVIATIONS.